

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет информационных технологий
Кафедра общей информатики

Направление подготовки: 09.03.01 Информатика и вычислительная техника
Направленность (профиль): Программная инженерия и компьютерные науки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Соломенникова Николая Александровича

Тема работы:

**РАЗРАБОТКА СИСТЕМЫ АВТОМАТИЗИРОВАННОГО ПОПОЛНЕНИЯ
ОНТОЛОГИЧЕСКОЙ МОДЕЛИ КАФЕДРЫ ЗНАНИЯМИ, ИЗВЛЕЧЕННЫМИ ИЗ
ДОКУМЕНТОВ**

«К защите допущена»
Заведующий кафедрой,
д.ф.-м.н., доцент
Пальчунов Д. Е. /.....
(ФИО) / (подпись)
«.....» мая 2026 г.

Руководитель ВКР
д.ф.-м.н., доцент
зав. каф. общей информатики ФИТ
Пальчунов Д. Е. /.....
(ФИО) / (подпись)
«.....» мая 2026 г.

Соруководитель ВКР

старший преподаватель кафедры общей
информатики ФИТ
Найданов Ч. А. /.....
(ФИО) / (подпись)
«.....» мая 2026 г.

Новосибирск, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)
Факультет информационных технологий

Кафедра общей информатики
(название кафедры)

Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль): Программная инженерия и компьютерные науки

УТВЕРЖДАЮ

Зав. кафедрой Пальчунов Д. Е.
(фамилия, И., О.)

.....
(подпись)

24.10.2025

ЗАДАНИЕ

НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА

Студенту Соломенникову Николаю Александровичу, группы 22204
(фамилия, имя, отчество, номер группы)

Тема Разработка системы автоматизированного пополнения онтологической модели кафедры знаниями, извлеченными из документов
(полное название темы выпускной квалификационной работы)

утверждена распоряжением проректора по учебной работе 0754 от 24.10.2025.

Срок сдачи студентом готовой работы 20 мая 2026 г.

Исходные данные (или цель работы):

Разработать систему автоматизированного пополнения онтологической модели кафедры знаниями, извлечёнными из документов

Структурные части работы:

Анализ предметной области, проектирование системы, реализация и апробация системы

Руководитель ВКР
зав. каф. общей информатики ФИТ,
д.ф.-м.н., доцент
Пальчунов Д. Е./.....
(ФИО) / (подпись)

24.10.2025

Задание принял к исполнению
Соломенников Н. А./.....
(ФИО студента) / (подпись)

24.10.2025

Соруководитель ВКР
старший преподаватель кафедры общей
информатики ФИТ

Найданов Ч. А./.....
(ФИО) / (подпись)

24.10.2025

СОДЕРЖАНИЕ

Определения, обозначения и сокращения	6
Введение	7
Основная часть	11
1 Анализ предметной области.....	11
1.1 Онтологические модели и их пополнение.....	11
1.1.1 Онтологии как способ формализации знаний.....	11
1.1.2 Двухуровневое устройство онтологии.....	12
1.1.3 Свойства, существенные для задачи	12
1.2 Документы кафедры и проблемы извлечения знаний	13
1.2.1 Документооборот и форматы.....	13
1.2.2 Проблемы извлечения знаний.....	14
1.3 Обзор подходов к извлечению знаний из документов	15
1.3.1 Правила-ориентированные методы.....	15
1.3.2 Методы на основе обучения.....	16
1.3.3 Методы на основе больших языковых моделей.....	16
1.3.4 Гибридные подходы	16
1.4 Существующие программные решения	17
1.5 Требования к разрабатываемой системе	19
1.5.1 Функциональные требования	19
1.5.2 Нефункциональные требования	20
1.5.3 Принятые ограничения.....	21
2 Проектирование системы	22
2.1 Архитектурная концепция.....	22
2.1.1 Разделение слоёв «данные» и «знания»	22
2.1.2 Шаблонно-ориентированное извлечение	23
2.1.3 Гибридная экстракция	23
2.2 Унифицированное представление документов	24
2.2.1 Назначение и принципы	24
2.2.2 Структура	24

2.3 Конвейер обработки документа	25
2.3.1 Общая схема	25
2.3.2 Преобразование к унифицированному представлению	26
2.3.3 Определение класса документа	27
2.3.4 Извлечение значений полей	28
2.3.5 Построение чернового графа фактов	28
2.3.6 Интеграция в онтологическую модель	29
2.4 Шаблоны как механизм экстракции	29
2.4.1 Состав шаблона	29
2.4.2 Программное представление шаблона	30
2.4.3 Меры по снижению рисков императивного подхода	30
2.5 Декларативный язык извлечения значений полей	31
2.5.1 Поле как единица извлечения	31
2.5.2 Селектор	32
2.5.3 Извлекатель	32
2.5.4 Нормализатор	33
2.6 Концепты и идентификация индивидов онтологии	34
2.6.1 Понятие концепта	34
2.6.2 Виды концептов и идемпотентность	34
2.6.3 Стратегии формирования IRI	34
2.6.4 Эволюционный пример: отказ от концепта «вид практики»	37
2.7 Граф фактов и его редактирование	37
2.7.1 Черновой граф	37
2.7.2 Двухслойная модель «черновик + правки»	38
2.7.3 Дополнительные факты и схема соотношения знаний	38
2.8 Интеграция знаний в онтологическую модель	38
2.8.1 Политики слияния	38
2.8.2 Эффективная дата документа	39
2.8.3 Согласование с существующими индивидами	40
2.8.4 Хранение и журналирование	40

2.9 Программная архитектура системы	40
3 Реализация и апробация системы	43
3.1 Технологический стек	43
3.2 Технические решения, заслуживающие отдельного рассмотрения	44
3.2.1 Контекст документа: ленивая загрузка ресурсов	44
3.2.2 Морфологическое приведение ФИО	45
3.2.3 Структурная валидация шаблона через статический анализ	46
3.2.4 Контракт ответа языковой модели	47
3.2.5 Политики слияния как декларативные аннотации схемы.....	47
3.2.6 Откат документа: история как источник истины для ABox	48
3.3 Пользовательский интерфейс	49
3.3.1 Вкладка документов и редактор графа фактов	49
3.3.2 Вкладка шаблонов.....	51
3.3.3 Вкладка онтологии и навигация по модели	52
3.3.4 Известные ограничения интерфейса.....	53
3.4 Апробация системы	53
3.4.1 Тестовый набор документов и проверенные сценарии.....	53
3.4.2 Качественная оценка результатов.....	54
3.5 Известные ограничения и направления развития	55
Заключение	57
Список использованных источников и литературы	59
Приложение А	62
Приложение Б	63
Приложение В	64
Приложение Г	65
Приложение Д	78

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ISO/IEC — Международная организация по стандартизации / Международная электротехническая комиссия.

W3C (World Wide Web Consortium) — консорциум организаций, вырабатывающий стандарты для технологий World Wide Web.

COM (Component Object Model) — компонентная объектная модель Microsoft для межпроцессного взаимодействия.

TEI (Text Encoding Initiative) — стандарт XML-разметки текстов, используемый в гуманитарных науках.

LLM (Large Language Model) — большая языковая модель.

IRI (Internationalized Resource Identifier) — расширение URI (Uniform Resource Identifier), допускающее использование символов Unicode; используется как идентификатор сущности в RDF и OWL.

Turtle (TTL) (Terse RDF Triple Language) — текстовый формат сериализации RDF-графов.

XSD (XML Schema Definition) — язык описания структуры XML-документов.

Класс онтологии — категория сущностей предметной области, описанная в схеме онтологии; экземпляры класса называются индивидами.

Индивид — конкретный объект предметной области, представленный в онтологии; связан с литералами и другими индивидами через свойства.

Свойство (предикат) — именованное отношение в онтологии; объектные свойства связывают индивидов между собой, свойства данных — индивидов с литералами.

Литерал — значение данных (строка, число, дата) в RDF-графе.

Аннотация — особый вид свойства OWL, используемый для описания терминов онтологии метаинформацией, не входящей в формальный логический вывод.

Идемпотентность — свойство объекта или операции при повторном применении давать тот же результат, что и при первом.

ВВЕДЕНИЕ

В условиях цифровизации образовательных и административных процессов в высших учебных заведениях возрастает объём документов, сопровождающих деятельность кафедр: заявлений обучающихся, индивидуальных заданий на практику, отчётов руководителей, ведомостей, служебных записок, тезисов и иных документов, содержащих значимые сведения о составе участников учебного процесса, академических активностях и их результатах. Несмотря на наличие этих данных, их практическое использование затруднено: информация представлена преимущественно в слабоструктурированном виде, не имеет формальной модели представления знаний и не интегрирована в единое хранилище.

Одним из перспективных подходов к решению этой проблемы является использование онтологических моделей предметной области. Онтология явно задаёт классы объектов, их свойства и отношения, что делает возможным выполнение запросов к накопленным знаниям, обнаружение связей между сведениями из разных источников и автоматизированный логический вывод. Однако наполнение онтологии актуальными данными, извлекаемыми из документов, на практике осуществляется вручную, что требует значительных трудозатрат, подвержено ошибкам и не масштабируется при увеличении объёма документов.

Существующие методы автоматизированного извлечения информации можно разделить на три группы: правила-ориентированные методы, методы на основе обучения с использованием обработки естественного языка [8], методы на основе больших языковых моделей [9]. Каждый из них в чистом виде имеет существенные ограничения: правила-ориентированные методы неустойчивы к вариативности оформления документов; методы на основе обучения требуют значительных по объёму размеченных корпусов, непрактичных в условиях ограниченного потока однотипных документов кафедры; методы на основе больших языковых моделей характеризуются ограниченной интерпретируемостью и риском генерации правдоподобных, но не

соответствующих действительности значений. Готовые программные решения, реализующие описанные методы, ориентированы преимущественно на извлечение текстового содержимого без привязки к конкретной онтологической модели; использование облачных сервисов интеллектуальной обработки документов [20–22] сопряжено с передачей данных внешним поставщикам, что в контексте обработки документов с персональными данными [1] накладывает существенные правовые ограничения.

Возникает актуальная научно-техническая задача — разработка системы, обеспечивающей автоматизированное извлечение данных из слабоструктурированных документов кафедры и их преобразование в формализованное представление, интегрируемое в онтологическую модель, с возможностью контролируемого ручного вмешательства и сохранением связи каждого факта с документом-источником.

Цель работы — разработка системы автоматизированного пополнения онтологической модели кафедры знаниями, извлечёнными из документов.

Для достижения цели поставлены следующие **задачи**:

1. провести анализ предметной области, видов документов кафедры, методических подходов к извлечению знаний и существующих программных решений;
2. сформулировать функциональные и нефункциональные требования к системе;
3. спроектировать архитектуру системы, включая унифицированную модель представления документов, конвейер обработки, механизм шаблонов извлечения, модель идентификации индивидов онтологии и политику интеграции знаний в общую модель;
4. реализовать разработанную систему с пользовательским графическим интерфейсом;
5. провести апробацию системы на реальных документах кафедры и проанализировать её результаты.

Объектом исследования является процесс извлечения и формализации знаний из документальных источников. **Предметом исследования** — архитектурные и методические решения, обеспечивающие автоматизированное пополнение онтологической модели на основе данных, полученных из документов учебно-организационной деятельности кафедры.

Научная новизна работы заключается в разработке нескольких взаимосвязанных решений, отсутствующих в известных аналогах:

- *концептно-ориентированной модели идентификации индивидов онтологии*, в которой каждому классу сущностей сопоставлен программный модуль, инкапсулирующий стратегию формирования идентификаторов и набор идентифицирующих литералов; модель обеспечивает воспроизводимое и согласованное отождествление одних и тех же сущностей реального мира между разными документами;
- *гибридной модели извлечения* «детерминированный шаблон плюс коррекция большой языковой моделью», в которой языковая модель используется как проверяющий механизм, а не первичный источник знаний, что одновременно обеспечивает воспроизводимость и устойчивость к вариативности оформления;
- *политик слияния значений предикатов*, заданных декларативно в схеме онтологии в виде аннотаций, что позволяет гибко настраивать поведение интеграции без изменения программного кода.

Практическая значимость работы состоит в создании программной системы, применимой на кафедре общей информатики факультета информационных технологий Новосибирского государственного университета. Система обеспечивает автоматизацию ранее выполнявшегося вручную процесса извлечения сведений из документов учебного процесса и их интеграции в общую модель, что снижает трудозатраты, повышает полноту и согласованность накопленных знаний и создаёт основу для последующей аналитической работы.

Методы исследования: анализ предметной области и существующих программных решений; проектирование архитектуры программной системы; синтез и реализация её ключевых компонентов; разработка формальной модели представления документов; апробация системы на реальных документах кафедры с качественной оценкой результатов.

Структура работы. Работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе анализируется предметная область, методические подходы и существующие программные решения, формулируются требования к системе. Во второй главе излагаются архитектурные и проектные решения, положенные в основу системы. В третьей главе описываются технические аспекты реализации, результаты апробации и направления дальнейшего развития.

ОСНОВНАЯ ЧАСТЬ

1 Анализ предметной области

Разработка системы автоматизированного пополнения онтологической модели кафедры требует предварительного анализа онтологических моделей и принципов их пополнения, специфики документов кафедры как источника знаний, существующих методических подходов к извлечению информации, готовых программных решений сходного назначения, а также формализации требований к проектируемой системе. В настоящей главе эти вопросы рассматриваются последовательно; сформулированные сведения служат основой для архитектурных решений последующих глав.

1.1 Онтологические модели и их пополнение

1.1.1 Онтологии как способ формализации знаний

Под онтологией в работе понимается формальное описание концептуализации предметной области, явно задающее классы объектов, их свойства и отношения, обеспечивающее семантическую интерпретацию данных и возможность логического вывода [4]. В качестве базового формализма применяется рекомендованный консорциумом W3C язык OWL (Web Ontology Language) [15], основанный на модели данных RDF (Resource Description Framework) [16]. RDF задаёт представление знаний в виде помеченных ориентированных графов, узлы которых соответствуют сущностям предметной области либо литеральным значениям, а помеченные дуги — отношениям между ними; атомарной единицей описания является *триплет* «субъект — предикат — объект». OWL расширяет RDF возможностями описания классов, иерархий, ограничений мощности и дизъюнктивности.

Применительно к образовательному подразделению вуза онтологическая модель позволяет систематизировать знания об учебном процессе: структуре подразделений, составе сотрудников и студентов, учебной траектории обучающихся, академических активностях (ВКР, практиках, конференциях). Такая модель служит единым формализованным источником знаний, доступным для машинной обработки, выполнения запросов и логических выводов.

1.1.2 Двухуровневое устройство онтологии

С практической точки зрения онтологическая модель разделяется на два уровня. *Схема онтологии* (в терминах дескриптивных логик [5] — TBox и RBox) включает определения классов сущностей, свойств, ограничений и аксиом, выражающих терминологические знания. *Данные онтологии* (ABox) содержат сведения о конкретных индивидах, их свойствах и отношениях, выражающие фактологические знания.

Разделение существенно для задачи автоматизированного пополнения. Схема — результат экспертной работы и закладывается заранее, до начала эксплуатации; автоматизированное извлечение схемы из документов невозможно: документы иллюстрируют применение терминологических утверждений, но не содержат их. ABox же регулярно обновляется естественным ходом учебного процесса: каждое новое событие (прибытие сотрудника, защита ВКР, окончание практики) приводит к появлению новых индивидов и фактов о них. Именно эти знания доступны автоматизированному извлечению.

Соответственно, в рамках задачи извлечению подлежат: индивиды как экземпляры классов онтологии; значения данных в виде литералов; факты в виде ABox-аксиом о принадлежности индивидов классам и о значениях их свойств. Документы рассматриваются как источник данных, а не источник схемы.

1.1.3 Свойства, существенные для задачи

При проектировании системы пополнения онтологии необходимо обеспечить несколько свойств.

Стабильность идентификации индивидов. Один и тот же реальный объект может упоминаться в разных документах по-разному (полным ФИО, фамилией с инициалами, фамилией с должностью). Если каждое упоминание породит отдельный индивид, целостность модели разрушится. Система должна обеспечивать совпадение идентификаторов у эквивалентных упоминаний независимо от формы.

Согласованность модели. Добавление новых фактов не должно нарушать ни синтаксической, ни семантической целостности модели — ни на уровне ограничений TBox, ни логических противоречий с уже накопленными фактами.

Управление противоречиями. Помимо логических противоречий, в реальных документах возможны *фактические* конфликты: новый документ может приносить отличные от уже хранящихся сведения (например, новый номер группы после перевода). Поведение системы при таких конфликтах должно быть детерминированным и управляемым.

Отслеживание происхождения фактов. Для проверки и ручной корректировки система должна сохранять сведения о том, из какого документа и каким образом получен каждый факт.

Эти свойства будут учтены при формулировке требований (1.5) и архитектурных решений главы 2.

1.2 Документы кафедры и проблемы извлечения знаний

1.2.1 Документооборот и форматы

Деятельность кафедры сопровождается обработкой значительного числа документов: заявлений обучающихся, индивидуальных заданий на практику, отчётов руководителей, приказов, методических и научных материалов. Документы создаются в текстовых редакторах и распространяются преимущественно в форматах DOC и DOCX (на открытом стандарте Office Open XML [2]) и PDF (стандарт ISO 32000-1 [3]). Структура определяется не формальной моделью данных, а визуальным оформлением, ориентированным на человеческое восприятие.

Анализ реального набора документов кафедры показывает, что, несмотря на отсутствие явной формализации, большинство документов обладает устойчивыми шаблонными признаками: заявления содержат типовые реквизиты, ведомости — табличные данные с фиксированным набором столбцов, отчёты — повторяющиеся разделы. Это принципиально для последующего выбора шаблонного подхода: документы одного типа поддаются шаблонной обработке. Вместе с тем конкретная реализация шаблонных признаков существенно

варьируется от документа к документу — меняются формулировки, порядок следования полей, способы оформления.

Форматы документов по-разному хранят информацию о структуре. *DOCX* [2] обеспечивает прямой доступ к внутренней структуре: абзацы, списки, таблицы, заголовки явно представлены элементами внутреннего XML-описания (формат *DOC*, технически отличаясь, содержит ту же информацию). *PDF* [3] содержит извлекаемый текст, но структура выражена неявно: отсутствует прямое указание на абзацы и элементы списков, границы таблиц приходится восстанавливать по координатам текстовых фрагментов. *Сканированные изображения документов* не содержат ни текста, ни структуры и требуют предварительного применения оптического распознавания символов и анализа макета страницы.

В рамках работы первоочередное внимание уделено первым двум группам форматов как составляющим подавляющее большинство документов кафедры; поддержка сканированных документов рассматривается как направление дальнейшего развития (см. главу 3), для которого в архитектуре заложены соответствующие возможности.

1.2.2 Проблемы извлечения знаний

Анализ реальных документов позволяет выделить несколько основных проблем:

- *разнородность форматов*: разные форматы требуют разных подходов на этапе извлечения, и это разнообразие должно быть скрыто от последующих шагов обработки;
- *отсутствие единообразия структуры внутри одного класса*: документы одного типа различаются по компоновке, и правила, привязанные к жёстким позициям элементов, неустойчивы;
- *контекстная зависимость интерпретации*: значение в ячейке таблицы интерпретируется иначе, чем то же значение в сплошном тексте;

- *опциональные и повторяющиеся элементы*: части документа могут отсутствовать либо встречаться в неопределённом числе экземпляров;
- *неоднозначность текстовых форм*: одна сущность в разных документах именуется по-разному (полное ФИО — фамилия с инициалами, полное наименование — аббревиатура).

Ключевой методический вывод: извлечение знаний непосредственно из исходных документов сильно затруднено — правила оказываются переплетены с особенностями конкретного формата, и автор вынужден поддерживать отдельный набор правил для каждого формата. Решение — этап *унификации представления документов*: приведение к единой модели данных, в которой сохранены текст и минимально необходимая структурная информация и устранены особенности форматов. На таком унифицированном представлении правила извлечения формулируются единожды. Этот вывод определяет одно из центральных архитектурных решений работы (см. главу 2).

1.3 Обзор подходов к извлечению знаний из документов

Задача автоматизированного извлечения структурированной информации из текстовых документов изучается в нескольких смежных направлениях: в извлечении информации (Information Extraction) [8], в обучении и пополнении онтологий (Ontology Learning and Population) [6, 7], в обработке естественного языка (Natural Language Processing). Существующие подходы можно объединить в четыре группы.

1.3.1 Правила-ориентированные методы

Правила-ориентированные (rule-based) методы — исторически наиболее ранний подход, основанный на явном описании правил выделения интересующих фрагментов в виде регулярных выражений, шаблонов над последовательностями токенов, паттернов над синтаксическими деревьями [8]. Преимущества — *воспроизводимость и объяснимость* (каждое значение прослеживается до конкретного правила) и отсутствие требования к размеченным данным; основной недостаток — *хрупкость* к вариативности

оформления. Тем не менее в задачах со структурированными документами одного типа методы остаются конкурентоспособными.

1.3.2 Методы на основе обучения

Применение методов машинного обучения и обработки естественного языка — распознавание именованных сущностей [8], извлечение отношений, классификация и сегментация документов — обеспечивает устойчивость к вариативности: обученная модель распознаёт сущности в документах, существенно отличающихся от обучающих. Главное препятствие применению к рассматриваемой задаче — *требование к объёму размеченных данных* (сотни и тысячи документов с аннотированными значениями); для документооборота кафедры с ограниченным потоком однотипных документов сбор такого корпуса непрактичен. Дополнительный недостаток — слабая контролируемость пользователем и ограниченная объяснимость отдельных извлечений.

1.3.3 Методы на основе больших языковых моделей

Большие языковые модели (LLM) [9] получают на вход документ и запрос на естественном языке и выдают ответ в произвольной или структурированной форме. Сильная сторона — *гибкость*: одна модель обрабатывает документы разных типов без специального обучения. Слабые стороны существенны: *ограниченная интерпретируемость* — причина ответа не поддаётся анализу через формальные правила; *риск конфабуляции* — генерации правдоподобных, но ошибочных значений; *зависимость от внешнего сервиса* и *стоимость обращений*. Эти ограничения делают неприемлемым применение языковой модели как единственного источника знаний в задаче пополнения онтологии: включение в формализованную модель сведений, не поддающихся проверке, противоречит цели формализации.

1.3.4 Гибридные подходы

Гибридные подходы сочетают преимущества групп. Применительно к задаче пополнения онтологической модели кафедры оправдан подход, в котором *детерминированные правила шаблона выполняют первичное извлечение, а языковая модель используется как проверяющий и корректирующий механизм.*

Такое распределение ролей сохраняет ключевые преимущества правил-ориентированного подхода (воспроизводимость, объяснимость, отсутствие требования к разметке) и компенсирует его основной недостаток — хрупкость к вариативности: модель закрывает ровно те случаи, в которых правила неточны.

Сравнительные характеристики подходов сведены в таблицу 1.

Таблица 1 — Сравнительная характеристика подходов к извлечению

Подход	Воспроизводимость	Объяснимость	Устойчивость к вариативности	Требование к размеченным данным
Правила-ориентированный	высокая	высокая	низкая	отсутствует
На основе обучения	средняя	низкая	высокая	высокое
На основе LLM	средняя	низкая	высокая	отсутствует
Гибридный	высокая	высокая	высокая	отсутствует

Методический вывод, лежащий в основе настоящей работы: оправдан гибридный подход, сочетающий шаблонное извлечение по детерминированным правилам с проверкой и коррекцией результатов посредством большой языковой модели.

1.4 Существующие программные решения

Анализ готовых программных средств охватил четыре категории.

Универсальные средства извлечения текста и метаданных. Наиболее представительное решение — **Apache Tika** [14, 17], открытый набор инструментов, поддерживающий более тысячи форматов и делегирующий разбор соответствующим внутренним библиотекам. Сильная сторона — широта поддержки форматов; ограничение — Tika ориентирована на извлечение содержимого, а не его *понимания*: возвращается плоский текст с минимальной

разметкой, без сохранения логической структуры (заголовков, списков, вложенных таблиц), и без средств семантической интерпретации. Tika может рассматриваться лишь как одно из возможных средств приведения документа к текстовому виду, но не как решение задачи пополнения онтологии.

Специализированные системы извлечения структуры из научных публикаций. GROBID [10, 18] — реализация на базе машинного обучения для извлечения и структурного разбора PDF-публикаций в структурированные документы XML/TEI. Каскад моделей последовательной разметки оперирует токенами разметки страницы и учитывает их визуальные и позиционные характеристики; поддерживается распознавание порядка 68 типов меток с заявленной точностью более 90 %. Применимость к задачам кафедры ограничена: модели обучены на корпусах *научных публикаций*, выходным форматом является обобщённое TEI-представление без привязки к конкретной онтологии, поддерживается только PDF.

Облачные сервисы интеллектуальной обработки документов. Google Cloud Document AI [20], Microsoft Azure AI Document Intelligence [21] и Amazon Textract [22] реализуют извлечение значений на основе предобученных моделей, оптимизированных для типовых задач (счета, квитанции, документы удостоверения личности, формы); часть сервисов поддерживает дообучение. Применимость к рассматриваемой задаче ограничена несколькими обстоятельствами: сервисы ориентированы на извлечение значений полей, а не построение *онтологических фактов*; использование подразумевает *передачу документов внешним поставщикам*, что в условиях обработки документов с персональными данными [1] юридически затруднительно; обращение к сервисам платное, а зависимость от их доступности противоречит требованию автономности; режим «чёрного ящика» затрудняет объяснение конкретных решений.

Исследовательские системы пополнения онтологий. Задача автоматизированного пополнения онтологий из текста изучается на протяжении более двух десятилетий [6, 7]. Karma [11, 19] — инструмент интеграции данных,

отображающий разнородные источники (базы данных, табличные файлы, веб-сервисы) на онтологию; KnowledgeStore [12] — инфраструктура хранения, объединяющая структурированные и неструктурированные данные. Применимость ограничена: эти системы интегрируют *уже структурированные* данные, а не извлекают их из слабоструктурированных документов; они рассчитаны на масштаб, превышающий типичные потребности кафедры; применение готовой системы вынуждает принять её архитектурные решения, не учитывающие специфики русскоязычных ФИО, отечественных кодов направлений подготовки и принятых на кафедре формулировок документов.

Обоснование собственной разработки. Ни одно из готовых решений не покрывает в полной мере существенных для задачи потребностей: работы с целевой онтологией предметной области; локального исполнения без передачи документов внешним сервисам; поддержки отечественной специфики (морфология русскоязычных ФИО, отечественные коды направлений, принятые на кафедре формулировки); возможности контролируемого ручного вмешательства; расширяемости на новые типы документов добавлением шаблона; прозрачности и объяснимости каждого извлечённого факта. В связи с этим в рамках работы обоснована *собственная разработка* специализированной системы.

1.5 Требования к разрабатываемой системе

На основании проведённого анализа сформулированы требования к проектируемой системе.

1.5.1 Функциональные требования

Система должна обеспечивать полный цикл обработки документа от загрузки исходного файла до записи извлечённых фактов в общую онтологическую модель кафедры, с сохранением промежуточных результатов для просмотра пользователем. Должны поддерживаться основные распространённые форматы документов кафедры — DOC, DOCX, PDF с текстовым слоем; архитектура должна допускать расширение списка форматов.

Все документы должны приводиться к единому *унифицированному представлению*, сохраняющему текст и минимально необходимую логическую структуру (абзацы, списки, таблицы) и устраняющему особенности конкретных форматов. Класс документа должен определяться автоматически на основе его содержимого; должна быть предусмотрена возможность ручного указания класса при неудаче автоматической классификации.

Извлечение значений полей должно сочетать *детерминированные правила шаблона* и *проверку результатов с привлечением языковой модели*; результаты обоих этапов должны сохраняться для аудита. Извлечённые значения должны *нормализовываться к канонической форме* (морфологическое приведение ФИО, ISO-форма дат, сопоставление текстовых форм наименований стандартизованным индивидам перечислений). По каноническим значениям должны *строиться факты онтологии* — индивиды, литералы и связи; стратегия идентификации индивидов должна обеспечивать совпадение идентификаторов для упоминаний одного объекта в разных документах.

Пользователь должен иметь возможность *проверить и скорректировать результат* автоматического построения через графический интерфейс — исключить ложноположительные факты, переопределить значения, добавить факты вне шаблона. Извлечённые факты должны *интегрироваться в общую модель* с учётом конфликтов с уже накопленными; поведение при конфликтах должно быть детерминированным и определяться семантикой каждого свойства. Для каждого факта должно сохраняться *происхождение* (документ, шаблон, дата добавления), доступное через интерфейс. Должна быть предусмотрена *помощь пользователю по созданию заготовок шаблонов* на основе образцов документов и схемы онтологии.

1.5.2 Нефункциональные требования

Воспроизводимость — повторная обработка одного документа должна давать совпадающий набор фактов в детерминированной части (воспроизводимость этапа коррекции языковой моделью определяется настройками модели). *Объяснимость* — каждый факт должен быть прослежен

до конкретного поля шаблона, а каждое поле — до фрагмента документа. *Расширяемость* — подключение нового типа документа выполняется добавлением шаблона; расширение онтологии новыми свойствами не должно требовать правки кода (кроме случаев принципиально новых типов сущностей). *Ремонтопригодность* — архитектура допускает независимую модификацию отдельных подсистем. *Безопасность исполнения пользовательского кода* — для шаблонов в виде программного кода должны быть предусмотрены меры против нежелательных операций (статический анализ, ограничение импортов). *Кроссплатформенность* — применимость как в Windows, так и в других распространённых операционных системах.

1.5.3 Принятые ограничения

При проектировании сознательно приняты ограничения, оправданные исходной задачей. Система ориентирована на *однопользовательский режим*; все данные хранятся *локально*; оценка качества носит *качественный, а не статистический характер* (формирование размеченного корпуса не предусматривается); *поддержка сканированных документов* в текущей версии не реализована (для неё заложен каркас внешних извлекателей данных, см. главу 2).

2 Проектирование системы

В настоящей главе излагаются архитектурные и проектные решения, положенные в основу разработанной системы: концептуальное разделение слоёв данных и знаний, модель унифицированного представления документов, конвейер обработки и его пять стадий, механизм шаблонов извлечения, декларативный язык извлечения значений полей, концептно-ориентированная модель идентификации индивидов онтологии, организация графа фактов и его редактирование, политика интеграции знаний в общую модель, программная архитектура системы.

2.1 Архитектурная концепция

2.1.1 Разделение слоёв «данные» и «знания»

Ключевая идея системы — явное разделение двух уровней обработки документа. Документ не интерпретируется как непосредственный источник онтологических фактов; между исходным файлом и общей моделью знаний помещается промежуточное представление, в котором сохранены текст и минимально необходимая логическая структура, но не присутствует семантика предметной области. Общая схема обработки приведена на рисунке 1: документ произвольного поддерживаемого формата сначала преобразуется в унифицированное иерархическое представление, а затем по этому представлению, по правилам шаблона данного класса документа, формируются формализованные факты и добавляются в онтологическую модель.

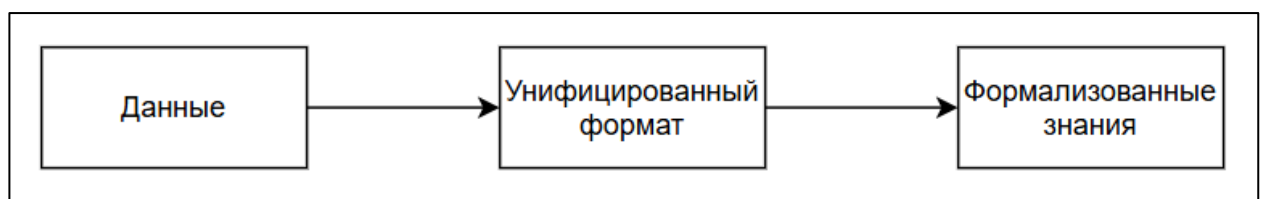


Рисунок 1 — Общая схема обработки документа

Введение промежуточного слоя даёт три важных свойства:

- *форматная независимость* — особенности конкретных форматов устраняются на этапе преобразования, и правила извлечения применяются единообразно ко всем документам одного типа;

- *прозрачность* — пользователь видит промежуточное представление и может сопоставить структуру документа с правилами шаблона;
- *правила-ориентированность* — более простое написание правил.

2.1.2 Шаблонно-ориентированное извлечение

Каждому типу документа кафедры (заявление о теме ВКР, индивидуальное задание на практику, отчёт руководителя, тезисы конференции и т. п.) ставится в соответствие отдельный *шаблон*, объединяющий три знания: критерии распознавания принадлежности документа классу, перечень извлекаемых полей и правила сборки фактов онтологии. Подход обоснован устойчивостью логической структуры документов одного типа, известной эксперту предметной области.

Полностью статистические методы [8] были отвергнуты как неприменимые в условиях ограниченного потока однотипных документов кафедры: они требуют размеченных обучающих корпусов, плохо контролируются пользователем и не гарантируют воспроизводимости. Готовые сервисы интеллектуальной обработки документов отклонены по тем же причинам, дополненным невозможностью явно задать семантику извлекаемых значений и связь с конкретной онтологией.

2.1.3 Гибридная экстракция

Шаблонные правила, заданные раз и навсегда, не покрывают всю вариативность документов одного типа. Поэтому в системе принят *гибридный подход*: детерминированное извлечение по правилам шаблона дополнено проверкой и коррекцией значений большой языковой моделью [9]. Модель получает документ целиком и предварительные значения полей и либо подтверждает их, либо исправляет, либо находит значение там, где правила шаблона не справились. Принципиально, что языковая модель используется как *проверяющий* механизм, а не первичный источник: основная нагрузка по извлечению лежит на детерминированной части, обеспечивающей воспроизводимость и объяснимость.

2.2 Унифицированное представление документов

2.2.1 Назначение и принципы

Унифицированное представление документа — промежуточная модель, сохраняющая текст и минимально необходимую логическую структуру и изолирующая последующую обработку от особенностей конкретных форматов хранения. Это не модель знаний и не формат хранения, а связующее звено между ними.

При проектировании учитывались следующие принципы. *Минимальность*: в модель входят только те элементы структуры, которые реально используются при извлечении знаний; сведения о шрифтах, секциях, колонтитулах, нумерации намеренно исключены — они не имеют устойчивой семантики между разными типами документов и лишь усложняют шаблонные правила. *Форматная независимость*: один и тот же документ в разных форматах должен давать одно и то же представление. *Иерархичность*: документ моделируется как дерево логического вложения, а не визуального расположения. *Детерминированность*: одни и те же входные данные неизменно дают одинаковое представление. *Ориентированность на шаблоны*: состав элементов подобран так, чтобы упростить запись правил поиска и навигации.

2.2.2 Структура

Документ представляется деревом (рисунок 2): корнем является сам документ, прямыми детьми корня и любого вложенного контейнера — блоки трёх типов.

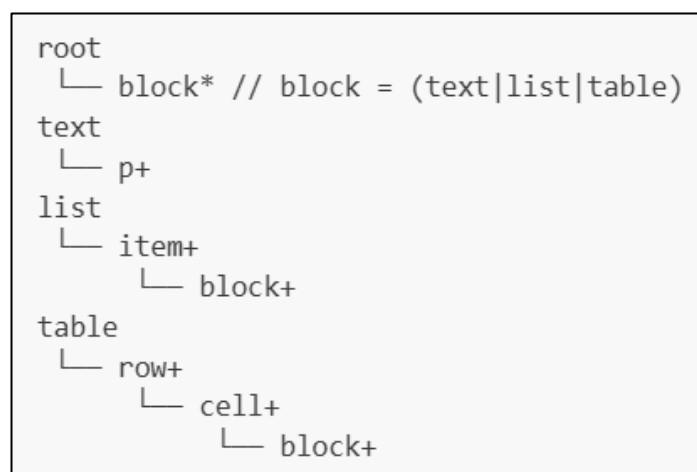


Рисунок 2 — Структура унифицированного представления документа

- *Текстовый блок* — последовательность абзацев, объединённых в одну логическую единицу (например, в пределах раздела документа).
- *Список* — последовательность *элементов*; каждый элемент может содержать вложенные блоки любого типа.
- *Таблица* — последовательность *строк*, каждая из строк содержит *ячейки*, каждая ячейка — вложенные блоки любого типа.

Единственным элементом, непосредственно несущим строку текста, является *абзац*. Остальные элементы — контейнеры. Терминологическое разграничение «абзац» / «текстовый блок» существенно: первое обозначает атомарный носитель текста, второе — контейнер связных абзацев.

Описанный набор элементов необходим (без него теряется контекст интерпретации значений в списках и таблицах) и достаточен (на нём описываются все формы представления знаний, встречающиеся в документах кафедры). Представление сериализуется в XML; структура формально описана XSD-схемой, по которой проверяется корректность результата преобразования (краткий пример документа в этом представлении — в приложении А).

2.3 Конвейер обработки документа

2.3.1 Общая схема

Обработка документа организована как последовательность из пяти стадий (рисунок 3, таблица 2). Каждая стадия принимает результат предыдущей и переводит документ из одного состояния в следующее.

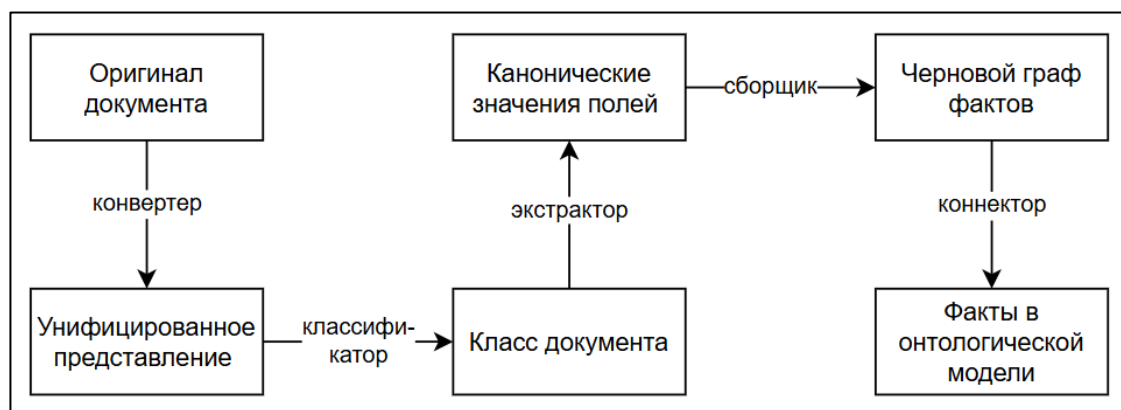


Рисунок 3 — Конвейер обработки документа

Таблица 2 — Стадии конвейера обработки документа

№	Стадия	Вход	Выход
1	Преобразование к унифицированному представлению	Исходный файл документа	Унифицированное представление
2	Определение класса документа	Унифицированное представление, имя файла	Идентификатор шаблона
3	Извлечение значений полей	Унифицированное представление, описание полей	Канонические значения полей с историей извлечения
4	Построение чернового графа фактов	Канонические значения, правила сборки шаблона	Черновой граф фактов
5	Интеграция в онтологическую модель	Черновой граф, ручные правки пользователя	Запись в общей модели и журнал слияния

Состояние документа явно фиксируется после каждой стадии в персистентном хранилище. Это обеспечивает *возобновляемость*: повторный запуск не выполняет уже пройденные стадии. Каждая стадия обрабатывает свои ошибки и переводит документ в стабильное состояние с поясняющим сообщением; падения с произвольной непредвиденной ошибкой исключены архитектурно. Пользователь может вмешаться в двух точках: указать класс документа явно (если автоматическая классификация отказала) и отредактировать черновой граф перед интеграцией.

2.3.2 Преобразование к унифицированному представлению

Первая стадия не интерпретирует семантику текста; её задача — сохранить текст и минимально необходимую структуру. Поскольку разные форматы хранят

информацию о структуре по-разному, в системе предусмотрены три типа преобразований:

- **нормализующие преобразования между форматами без потерь** применяются рекурсивно для приведения документа к удобному для разбора формату (например, DOC → DOCX или PDF → DOCX);
- **внутренние разборщики структуры** непосредственно строят унифицированное представление из документа известного формата (для DOCX — собственный обходчик внутренней XML-структуры, распознающий абзацы, списки, таблицы и заголовки);
- **внешние извлекатели данных** — каркас для подключения сторонних систем интеллектуальной обработки и оптического распознавания символов; в текущей версии реализован, но конкретных интеграций не имеет.

По завершении проверяется соответствие результата XSD-схеме; документы, не прошедшие проверку, дальше по конвейеру не передаются.

2.3.3 Определение класса документа

Под *классом/типом документа* понимается группа документов, обладающих общей структурой и семантикой и обрабатываемых единым шаблоном. Каждому классу соответствует ровно один шаблон. Классификатор перебирает известные шаблоны, обращаясь к каждому с вопросом о применимости; шаблон самостоятельно определяет принадлежность по имени файла и унифицированному представлению. Возвращается первый подошедший шаблон.

Альтернатива — сравнение структуры документа с эталонной структурой шаблона — отвергнута: документы одного класса нередко существенно различаются по компоновке, и автору удобнее указать в шаблоне именно те признаки, которые устойчивы для данного типа. При неудаче автоматической классификации пользователь указывает класс вручную; это рассматривается как штатный сценарий.

2.3.4 Извлечение значений полей

Стадия состоит из трёх последовательных этапов и реализует центральную идею гибридной экстракции (см. 2.1.3).

Этап декларативного извлечения. Для каждого поля запускается объявленные в составе поля селектор и извлекатель (см. 2.5). Результат сохраняется либо как извлечённое значение, либо как причина отказа.

Этап проверки и коррекции языковой моделью. Один запрос на документ передаёт модели унифицированное представление целиком, список полей с описаниями и результаты предыдущего этапа. Модель действует по строго формализованному контракту: подтверждает, исправляет или находит значение заново. Объединение проверки и поиска в одной точке оправдано тем, что модель, видя сразу все поля и весь документ, использует их взаимный контекст лучше, чем при поочерёдных независимых запросах.

Этап нормализации. Сырое значение (с приоритетом коррекции модели) пропускается через нормализатор, специфичный для поля. Результат — каноническая строка либо отказ с поясняющей причиной. Эта строка и есть то, с чем работает построитель графа.

Разделение этапов чётко разграничивает ответственность: за форму отвечает декларативный этап, за смысл — модель, за привязку к онтологии — нормализатор. По каждому полю в любой момент можно установить, кем и как было получено итоговое значение.

2.3.5 Построение чернового графа фактов

Четвёртая стадия впервые оперирует онтологическими сущностями — индивидами и литералами. Построение выполняется *императивной частью шаблона*: автор, опираясь на знания предметной области, описывает, какие факты порождаются из извлечённых значений. Шаблон обращается к *построителю графа* — программному интерфейсу, скрывающему работу с RDF за операциями уровня предметной области (см. 2.7).

Принципиальное свойство стадии: построитель *никогда не выбрасывает исключений*. Любая ошибка (пустое поле, отвергнутое концептом значение,

рассогласование с онтологией) превращается в *неполный узел* с поясняющим сообщением и ссылкой на исходное поле. Граф остаётся синтаксически корректным, и пользователь сразу видит в интерфейсе, какие именно триплеты не построились и почему.

Все ручные правки чернового графа при перезапуске стадии сбрасываются: правки имеют смысл только относительно конкретной версии черновика, и автоматическое наложение прежних правок на изменённый черновик привело бы к непредсказуемым результатам.

2.3.6 Интеграция в онтологическую модель

Пятая стадия объединяет факты документа с общей моделью. К черновому графу применяются ручные правки пользователя; подмешиваются дополнительные факты, добавленные пользователем вне шаблона (см. 2.7.3); опционально выполняется согласование IRI индивидов с уже существующими в модели (см. 2.8.3); определяется *эффективная дата документа* по приоритетному списку полей дат; происходит слияние с общей моделью по политикам, заданным для каждого предиката (см. 2.8.1). Перед слиянием проверяется целостность текущей модели; при обнаружении нарушений интеграция блокируется. Все события слияния — добавление, отзыв, отклонение факта — фиксируются в журнале.

2.4 Шаблоны как механизм экстракции

2.4.1 Состав шаблона

Шаблон — программная сущность, формализующая правила обработки одного класса документов. Он объединяет три равноправных части:

- *правила распознавания принадлежности* — функция, получающая имя файла и унифицированное представление и возвращающая признак применимости;
- *описание извлекаемых полей* — декларативный список объектов *поле*, каждое из которых описывает одну атомарную извлекаемую ценность через имя, смысловое описание, селектор, извлекатель и нормализатор (см. 2.5);

- *правила сборки графа* — императивная функция, получающая построитель и канонические значения полей и формирующая факты онтологии.

Объединение трёх знаний в одной сущности отражает естественную связку: документ относится к классу тогда и только тогда, когда из него извлекаются объявленные поля и они собираются в граф предметной области.

2.4.2 Программное представление шаблона

Шаблон представлен программным модулем на языке общего назначения с фиксированным интерфейсом. Альтернативные подходы — внешний предметно-ориентированный язык, декларативные шаблоны на YAML/JSON, шаблоны на онтологических запросах — отвергнуты: каждый из них в итоге упирается в потолок выразительности при попытке описать вариативность реальных документов.

В пользу программного представления говорят: высокая выразительность, отсутствие необходимости изучения нового языка, возможность повторного использования общих функций, поддержка инструментами разработки. Пример кода шаблона — в приложении Б.

2.4.3 Меры по снижению рисков императивного подхода

Исполняемый пользовательский код несёт известные риски: сложность статического анализа, возможность нежелательных операций, ошибки на редких входных данных. В качестве компенсации предусмотрена *структурная валидация* шаблона по шести категориям перед его использованием:

1. *структурная корректность* — реализованы ли все методы фиксированного интерфейса;
2. *безопасность* — статический анализ исходного кода: запрет на опасные импорты (доступ к файловой системе, сеть, запуск подпроцессов), на опасные встроенные функции и магические атрибуты;
3. *корректность списка полей* — формат имён, отсутствие дубликатов, наличие осмысленных описаний;
4. *проверка распознавания класса* — пробный вызов на пустом представлении;

5. *сухой прогон сборки графа* — пробное построение с синтетическими значениями всех полей;

6. *согласованность с онтологией* — сверка обращений к терминам онтологии с фактической схемой, с подсказкой ближайшего по написанию имени при опечатках.

Отчёт о валидации с разделением замечаний по уровням важности доступен пользователю. Отдельной программной песочницы исполнения шаблонов в текущей версии нет: безопасность обеспечивается статическим анализом, что оправдано доверенным окружением кафедры.

2.5 Декларативный язык извлечения значений полей

2.5.1 Поле как единица извлечения

Поле — именованная декларация одной атомарной ценности, извлекаемой из документа. Оно описывается пятью характеристиками: уникальным именем в формате `snake_case`, смысловым описанием на естественном языке (используется языковой моделью при проверке и коррекции), селектором (декларативный путь по дереву документа), извлекателем (преобразование текста в сырую строку), нормализатором (приведение к канонической форме).

Эта пятёрка — единица всех трёх этапов извлечения: по полю работают декларативные правила, по полю формируется запрос к модели, по полю применяется нормализатор; статусы и ошибки также считаются по полю независимо. Жизненный цикл значения приведён на рисунке 4.

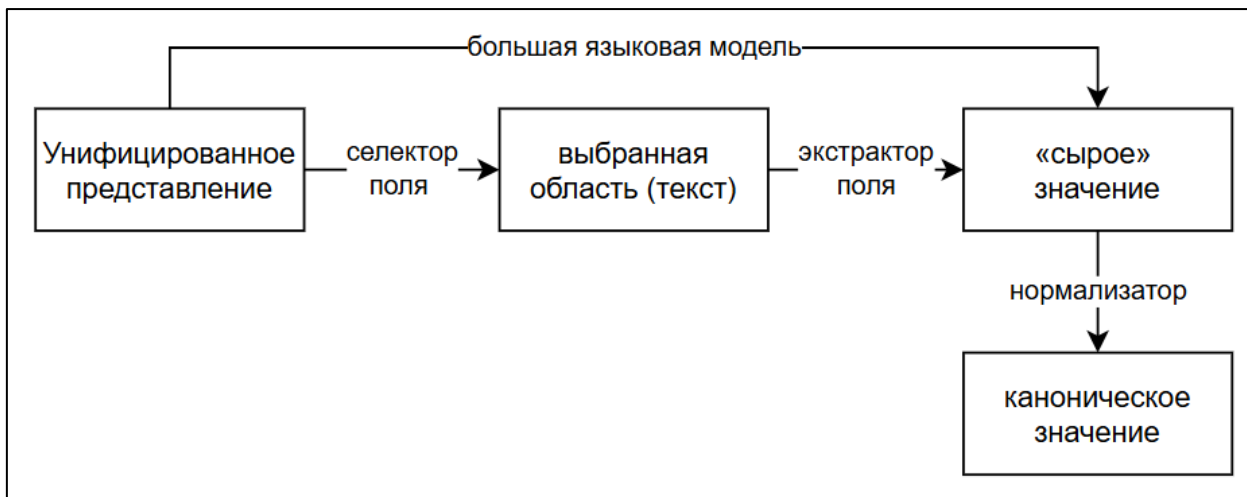


Рисунок 4 — Жизненный цикл значения поля

Одно поле может через концепт онтологии (см. 2.6) породить сразу несколько фактов: например, поле, содержащее ФИО, порождает индивид персоны и набор связанных литералов (фамилия, имя, отчество). Поле не отождествляется со свойством онтологии — это понятия разного уровня.

2.5.2 Селектор

Селектор реализует декларативное сужение области поиска по дереву документа цепочкой операций: первая применяется ко всему дереву, каждая следующая — к результату предыдущей. Доступные операции сгруппированы по назначению:

- *поиск элементов* по типу и условию (предикаты по содержимому текста, регулярному выражению, длине и их логические комбинации);
- *навигация* по соседним, вложенным и внешним элементам — позволяет задавать значение относительно якорного элемента (например, «ячейка справа от ячейки с заголовком “Группа”»);
- *выбор из множества* — первый, последний или элемент с указанным индексом;
- *срез между маркерами* — все элементы между двумя якорями;
- *пользовательская операция* для редких случаев.

Если в конце цепочки остаётся несколько кандидатов, берётся первый. Преимущество декларативного подхода перед прямыми регулярными выражениями и обходами дерева — *читаемость и совместимость со статическим анализом*: набор операций ограничен и документирован.

2.5.3 Извлекатель

Извлекатель преобразует текст выделенной области в сырую строку последовательностью операций. Каждая операция получает строку и возвращает новую строку либо признак неудачи (извлечение прерывается). Доступны:

- *выделение части* — по регулярному выражению с указанием группы, относительно маркера (после, до, между двумя маркерами);

- *строковые преобразования* — замена подстроки, схлопывание пробельных символов, обрезка по краям, изменение регистра;
- *фильтрация символов* — оставить только буквы и пробелы, цифры и спецсимволы, символы по регулярному выражению;
- *очистка шаблонных подписей* — удаление типичных меток бланков («Ф.И.О.», «Дата:», длинные подчёркивания, многоточия);
- *выбор по приоритету* — попробовать несколько регулярных выражений и взять первое сработавшее.

Извлекатель не выполняет онтологически-зависимых преобразований (приведение к именительному падежу, проверка формата даты, поиск канонического написания должности) — это работа нормализатора.

2.5.4 Нормализатор

Нормализатор — последовательность правил, приводящих сырое значение к канонической форме. Главная роль — *единственная точка соединения поля с предметной областью*: через нормализатор поле получает связь с конкретным концептом онтологии. Делегирование концепту записывается одним правилом «нормализовать как <концепт>»; вся сложность парсинга, морфологии и канонизации остаётся внутри концепта.

Для редких полей без концепта-аналога нормализатор предоставляет примитивные правила: целое число, число в диапазоне, проверка по регулярному выражению, ограничения на длину и количество слов, преобразования регистра, замена подстроки.

Описанная организация пришла на смену прежней двухуровневой схеме «валидатор + трансформер», в которой проверка формата и приведение к канонической форме были разнесены и могли вступать в противоречие (валидатор сообщал «ФИО не в именительном», а трансформер ниже по конвейеру всё равно исправлял). Объединение проверки и преобразования в одной операции устраняет это противоречие.

2.6 Концепты и идентификация индивидов онтологии

2.6.1 Понятие концепта

Концепт — программный модуль, инкапсулирующий полное знание об одном элементе онтологии: разбор сырого значения, приведение к канонической форме, определение локального имени IRI индивида (для классов индивидов), формирование набора идентифицирующих литералов. Концепт отделён от шаблонов: шаблон лишь указывает, какой концепт применить; вся специфика разбора и сборки скрыта внутри концепта.

Это даёт *согласованность* (одни и те же правила во всех шаблонах), *централизованное обновление* и *повторное использование*.

Терминологическое замечание: в работе термин «концепт» употребляется именно в программном смысле — как модуль с identity-стратегией; для соответствующего ему элемента схемы онтологии используется термин «класс онтологии». В литературе по дескриптивным логикам [5] и OWL [15] концептом нередко называется сам класс онтологии, и без явного разграничения возникает путаница.

2.6.2 Виды концептов и идемпотентность

Концепты делятся на два вида. *Концепты индивидов* соответствуют классам онтологии и при регистрации в графе порождают индивид со своим IRI и набор идентифицирующих литералов. *Концепты типов данных* существуют в графе только как литералы заданного типа (например, дата в ISO-формате с типом `xsd:date`).

Контракт концепта требует *идемпотентности парсинга*: повторное применение разбора к канонической форме даёт тот же канонический результат, $\text{canon}(\text{parse}(\text{canon}(\text{parse}(x)))) = \text{canon}(\text{parse}(x))$. Без этого пропадает воспроизводимость: тот же документ при повторном проходе мог бы породить индивиды с другими идентификаторами и не схлопнуть их с уже имеющимися.

2.6.3 Стратегии формирования IRI

Реализованы четыре стратегии идентификации, каждая под класс сущностей.

Хеш от нормализованных идентифицирующих признаков — для сущностей с неоднозначной текстовой идентификацией (персон, организаций, профилей подготовки). Признаки приводятся к нормализованному виду, IRI формируется как «префикс класса + короткий хеш ключа»; сами признаки параллельно сохраняются как литералы. Состав ключа подобран как компромисс между разрешающей способностью и устойчивостью к различиям в оформлении: для персон, например, в ключ входят фамилия полностью и только инициалы имени и отчества — чтобы упоминания «Иванов Иван Иванович» и «Иванов И. И.» объединялись в одного индивида.

Стабильный технический идентификатор — для сущностей с устойчивым «техническим» идентификатором (номер группы, код направления подготовки). IRI строится напрямую из значения с допустимыми безопасными преобразованиями — без хеширования. Это упрощает чтение графа человеком.

Именованный индивид перечисления — для закрытых списков (должности, учёные степени, учёные звания, итоговые оценки). Список зафиксирован в схеме онтологии; задача концепта — отобразить произвольное входное написание (включая сокращения и падежные формы) на одно из канонических наименований через таблицу синонимов.

Композитная идентификация — для *слабых сущностей* (событий, не имеющих собственного идентификатора, но определяемых связкой с другими сущностями). В системе таких два: выпускная квалификационная работа идентифицируется хешем от IRI студента-автора (одна работа на студента); практика — хешем от пары «студент, дата начала» (две практики одного студента не могут начинаться в один день). Без композитной идентификации разные документы об одном студенте схлопывали бы в один индивид по сути разные события.

Сводный перечень концептов и их стратегий приведён в таблице 3.

Таблица 3 — Концепты онтологии и стратегии идентификации

Концепт	Класс онтологии	Стратегия
Персона	Персона	Хеш от «фамилия
Организация	Организац ия	Хеш от нормализованного названия
Учебная группа	Группа	Стабильный идентификатор: префикс плюс номер группы
Направление подготовки	Направлен иеПодготов ки	Стабильный идентификатор: код с заменой точек на подчёркивания
Профиль подготовки	Профиль	Хеш от нормализованного названия
Выпускная работа	ВКР	Композитная: хеш от IRI студента
Практика	Практика	Композитная: хеш от пары (IRI студента, дата начала)
Должность	Должность	Именованный индивид перечисления
Учёная степень	УченаяСте пень	Именованный индивид перечисления
Учёное звание	УченоеЗва ние	Именованный индивид перечисления
Итоговая оценка	Оценка	Именованный индивид перечисления
Дата	(xsd:date)	Литерал в ISO-формате; индивид не порождается
Электронная почта	(xsd:string)	Нормализованный литерал; индивид не порождается

2.6.4 Эволюционный пример: отказ от концепта «вид практики»

Изначально вид практики был оформлен как концепт-перечисление с закрытым списком значений (учебная, производственная, научно-исследовательская работа, эксплуатационная, преддипломная). В реальных документах формулировки типа «производственной практики (научно-исследовательской работы)» в родительном падеже не сопоставлялись с синонимами; выявление всех падежных форм оказалось непропорционально трудоёмким и не давало гарантии покрытия будущих документов.

Принято решение об отказе от закрытого словаря: вид практики хранится как свободная строка с типом `xsd:string` и политикой слияния «добавление». Это упрощает шаблон и не теряет информации — фильтр по виду практики на запросах к графу выполняется подстрочно. Эпизод иллюстрирует общий принцип: *ремонтпригодность модели предпочтительнее её формальной красоты.*

2.7 Граф фактов и его редактирование

2.7.1 Черновой граф

Прямое использование стандартной модели RDF на стадии сборки было бы избыточным: она оперирует только полностью построенными триплетами, и при отсутствии значения построитель должен либо пропускать факты, либо подставлять заглушки. По этой причине введён собственный *черновой граф*, допускающий *неполные узлы*. Каждый узел несёт дополнительные метаданные: тип (IRI или литерал), значение (если построилось), причину отсутствия (если нет), *поле-источник* — имя поля шаблона, из которого получен узел.

Решение даёт три выигрыша одновременно. Граф можно сериализовать в любом состоянии, не теряя частичный результат при ошибке. Неполный узел сразу показывает пользователю, *что* не извлеклось. Указание поля-источника указывает, *где* искать причину. При полном графе он детерминированно преобразуется в стандартную модель RDF, которая и поступает в коннектор.

2.7.2 Двухслойная модель «черновик + правки»

Пользователь должен исправлять результат шаблона перед интеграцией; однако прямое редактирование чернового графа теряло бы правки при повторной сборке. Поэтому применена двухслойная модель: *черновой граф* перезаписывается при каждом успешном прогоне шаблона и непосредственно не редактируется; параллельно ведётся *слой правок*, содержащий индексы исключённых триплетов и переопределения отдельных узлов. Итоговый граф для интеграции получается наложением слоя правок на черновик.

Слои сохраняются в разных файлах. При повторной автоматической сборке правки не теряются и применяются к новой версии черновика — в той мере, в которой согласованы с ней. Этот подход иногда называют *override-редактированием*: автоматическая повторяемость сосуществует с возможностью точечного ручного вмешательства.

2.7.3 Дополнительные факты и схема соотношения знаний

Когда шаблон в принципе не покрывает некоторое знание из документа, дописывать его в шаблон ради единичного факта избыточно. Для таких случаев предусмотрена «отдушина» — самостоятельный TTL-файл с дополнительными фактами, который подмешивается в итоговый граф на стадии интеграции.

Знания можно разделить на два множества: *знания в документе* и *знания, извлекаемые шаблоном*. Их пересечение — факты, которые шаблон извлекает корректно; «только в документе» — знания, которые шаблон не покрывает (добавляются через дополнительные факты); «только из шаблона» — ложноположительные или устаревшие факты (исключаются через слой правок). Это удобный образ, поясняющий, что слой правок и дополнительные факты — не «недоработка», а архитектурная норма.

2.8 Интеграция знаний в онтологическую модель

2.8.1 Политики слияния

Поведение слияния не описывается единым правилом: один и тот же предикат может вести себя по-разному в зависимости от семантики. Номер

учебной группы — единственное актуальное значение; телефон — множественное; тема ВКР — изменяющееся во времени.

В системе поведение слияния задаётся *не в коде модуля интеграции, а в самой схеме онтологии* (приложение В) — аннотацией политики слияния, прикреплённой к каждому свойству. Поддерживаются три политики (таблица 4).

Таблица 4 — Политики слияния значений предикатов

Политика	Поведение	Применяется к
Полная замена	Single-valued, неизменное: новый факт полностью замещает существующие для (s, p)	Идентифицирующим свойствам и связям
Замена по дате	Single-valued, временное: новый факт замещает существующее только при бóльшей эффективной дате	Свойствам состояния во времени (группа, тема ВКР)
Добавление	Multi-valued: новый факт добавляется без замещения	Множественным свойствам (телефоны, звания); по умолчанию

Преимущество подхода: логика слияния описана в ТВох, а не «зашита» в коде; добавление новых свойств не требует доработки модуля интеграции — новое свойство автоматически получает политику по умолчанию.

2.8.2 Эффективная дата документа

Чтобы политика «замены по дате» могла принимать решения, каждому документу требуется *эффективная дата* — момент, к которому относятся его факты. Это не дата создания файла и не дата поступления в систему: для заявления — это дата подписания, для индивидуального задания — дата выдачи, для приказа — дата издания.

Эффективная дата вычисляется по приоритетному списку полей дат (дата подачи заявления, дата выдачи задания, дата протокола, дата приказа, дата начала практики, общая дата документа). Жёсткое фиксирование одного-единственного поля даты в шаблоне было бы менее устойчивым: при неудачном извлечении

основной даты вся интеграция становилась бы невозможной. Если не извлечена ни одна дата, факт в политике «замены по дате» рассматривается как самый ранний и может быть вытеснен любым датированным.

2.8.3 Согласование с существующими индивидами

Хеширование от идентифицирующих признаков работает в большинстве случаев, но возможны ситуации, когда один реальный объект упомянут в разных документах в несовместимых формах — например, полным наименованием организации в одном случае и аббревиатурой в другом. Для таких случаев перед слиянием выполняется опциональное *согласование IRI*: извлечённый граф проверяется на наличие в общей модели индивидов с совпадающими идентифицирующими литералами; при однозначном совпадении IRI индивида в извлечённом графе перенаправляется на канонический IRI существующего индивида.

Согласование выполняется консервативно: при множественности кандидатов переписывание не производится, ситуация фиксируется в журнале. Поддерживаются персоны, организации, профили подготовки и ВКР.

2.8.4 Хранение и журналирование

Источником истины для всего ABox служит *упорядоченная история документов* — список добавленных документов с указанием идентификатора, шаблона, эффективной даты и пути к фрагменту фактов. Материализованная модель и журнал событий слияния — *производные индексы*, при необходимости полностью пересобираемые из истории. Сборка модели выполняется лениво, с кэшированием по отпечатку истории.

Переобработка документа реализована как «удалить запись из истории и заново выполнить слияние». Перед каждой операцией выполняется контроль целостности, что обеспечивает корректность повторных проходов: старые факты удаляются, новые встают на их место с учётом политик и эффективных дат.

2.9 Программная архитектура системы

Описанные решения объединяются в архитектуру из нескольких слоёв.

- **Слой описаний данных** определяет структуры, передаваемые между слоями (документ, шаблон, результат извлечения, черновой граф, слой правок). Не содержит бизнес-логики.

- **Слой ядра** — переиспользуемые средства, доступные коду шаблонов: модель документа, декларативный язык извлечения, концепты, построитель графа, контракт шаблона и его валидация. Самый стабильный слой: его изменения вынуждают пересматривать существующие шаблоны.

- **Слой модулей конвейера** содержит реализации пяти стадий. Каждый модуль зависит от ядра и описаний данных, но не от других модулей; их последовательность задаёт оркестрация.

- **Слой работы с файлами** — менеджеры документов и шаблонов, репозиторий онтологии. Единственная точка контакта с файловой системой; замена реализации позволяет перейти на иной способ хранения без переработки прочих слоёв.

- **Слой оркестрации** связывает остальные слои в единое приложение: общий контекст, конвейер, обращение к языковой модели, настройки, журналирование.

- **Слой пользовательского интерфейса** — графический интерфейс на PySide6. Опирается на остальные слои, но никем не используется, что допускает работу системы без интерфейса (например, в скриптах).

Особенность архитектуры — ядро спроектировано как программный интерфейс, доступный коду шаблонов. Автор шаблона получает полный декларативный язык извлечения и набор концептов, но не имеет прямого доступа к модулям конвейера или хранилищу. Это разграничение защищает систему от случайного нарушения контрактов и одновременно даёт авторам шаблонов всю необходимую выразительность.

Параллельно с общим контекстом приложения вводится *контекст документа* — рабочая обёртка, в которой во время прохождения конвейера хранятся тяжеловесные промежуточные данные обработки одного документа.

Контекст загружает данные лениво и автоматически освобождает их по выходу из обработки, что снимает нагрузку на память при обработке большого числа документов; устройство контекста подробно описано в главе 3.

3 Реализация и апробация системы

В настоящей главе рассматриваются конкретные технические аспекты воплощения архитектурных решений главы 2: технологический стек, отдельные решения реализации, заслуживающие специального разбора, организация графического интерфейса, результаты апробации системы на реальных документах кафедры и направления дальнейшего развития.

3.1 Технологический стек

Перечень основных программных средств, использованных при реализации, сведён в таблицу 5.

Таблица 5 — Использованные программные средства

Назначение	Средство	Версия
Язык реализации	Python [23]	3.11
Графический интерфейс	PySide6 (Qt 6) [24]	6.x
Работа с RDF	rdflib [25]	7.x
Разбор DOCX	стандартные xml.etree, zipfile	—
Преобразование DOC → DOCX	LibreOffice (soffice) либо Microsoft Word через COM	—
Преобразование PDF → DOCX	Веб-сервис ConvertAPI	—
Морфология русского языка	py morphology3 [13]	1.x
Большая языковая модель	OpenAI SDK [26]; модель gpt-5.4-mini (по умолчанию)	—
Сериализация структур данных	pydantic, стандартный json	—

Использование **Python 3.11** в качестве и языка приложения, и языка шаблонов снимает необходимость в собственном предметно-ориентированном

формате и снижает порог входа для авторов шаблонов. Экосистема языка покрывает все ключевые подзадачи зрелыми библиотеками.

rdflib [25] используется только как внутренний контейнер итогового RDF-графа документа. Сборка графа в шаблоне идёт не через **rdflib**, а через собственный построитель **TemplateGraphBuilder**, скрывающий работу с триплетами за операциями уровня предметной области (см. 2.7). Это сократило набор операций, доступных автору, до минимально необходимого и устранило типичные при прямой работе с **rdflib** ошибки (неконсистентные пространства имён, неправильно типизированные литералы).

pymorphy3 [13] применяется концептом **PersonConcept** для приведения частей ФИО к именительному падежу (см. 3.2.2).

Преобразование документов формата **DOC** к формату **DOCX** [2] выполнено через цепочку **fallback**: первой попыткой запускается **LibreOffice** (**soffice --headless**), при его отсутствии — **Microsoft Word** через **COM**-интерфейс (через **pywin32**). Цепочка выбрана так, чтобы преобразование работало без специальной настройки: на машинах сотрудников, как правило, установлен **Microsoft Office**; на машинах разработчика и в автоматизированных средах — **LibreOffice**.

В качестве **большой языковой модели** по умолчанию используется **gpt-5.4-mini** через официальный **SDK OpenAI** [26]. Имя модели — константа настроек, заменяется без правки кода. Выбор обоснован соотношением качества и стоимости: для задачи проверки уже извлечённых шаблоном значений не требуется наиболее крупная модель, а задержки и затраты заметно ниже.

3.2 Технические решения, заслуживающие отдельного рассмотрения

3.2.1 Контекст документа: ленивая загрузка ресурсов

Конвейер оперирует разнородными по объёму данными: для крупного документа унифицированное представление, результат извлечения полей, черновой граф и загруженный код шаблона суммарно занимают существенный объём памяти. Передача этих данных через все уровни вызовов в виде явных

параметров затрудняла бы сигнатуры функций и приводила бы к загрузке заведомо ненужных данных.

Для решения задачи введён программный объект — *контекст документа* `DocumentContext` — с ленивой загрузкой данных по требованию и автоматическим освобождением по завершении обработки. Каждое тяжёлое поле объявлено парой свойств `@property` и `@property.setter`: при первом обращении выполняется чтение из файла, при последующих — возврат закэшированного значения, при присваивании — замена кэша. Жизненный цикл всей обработки документа оборачивается менеджером контекста.

Использование на уровне конвейера сводится к одной обёртке `with`, после выхода из которой все тяжёлые объекты автоматически освобождаются. Решение даёт сразу прозрачность сигнатур (модули конвейера принимают единственный аргумент `ctx`), экономию памяти (чтение файлов происходит только при первом обращении к соответствующему полю) и гарантированное освобождение ресурсов независимо от пути выхода — включая исключения и ранние `return`.

3.2.2 Морфологическое приведение ФИО

Концепт `PersonConcept` отвечает за идентификацию индивидов класса *Персона*. `IRI` вычисляется как хеш от ключа «фамилия в нижнем регистре + инициал имени + инициал отчества»; все три части предварительно приводятся к именительному падежу — без этого упоминания «Соломенникову Николаю Александровичу» в дательном падеже и «Соломенников Николай Александрович» в именительном получили бы разные идентификаторы.

Морфологический разбор выполняется средствами `rumorphy3` [13]. Прямое применение разбора, однако, ошибочно: для большинства частей ФИО анализатор возвращает несколько вариантов разбора, и без фильтра выбирается случайный омонимичный. Например, «Соломенников» без фильтра читается как множественное число существительного «соломенник» и приводится к «Соломенники». Чтобы этого избежать, из разборов отбираются только варианты с тегами фамилии, имени или отчества.

Дополнительно учитывается *омонимия по полу*: «Соломенникова Николая» — родительный падеж мужского имени, но та же фамилия в начальной форме может принадлежать женщине; «Иванова» может быть и женской формой «Иванов», и фамилией Ивановой. Чтобы предпочесть согласованный по полу вариант, перед нормализацией фамилии и отчества определяется пол по имени, и при сортировке разборов варианту с соответствующим полом отдаётся приоритет.

Идемпотентность достигается тем, что слова, уже находящиеся в именительном падеже, попадают в фильтр типизированных разборов и возвращаются без изменений. Для повторных проходов это критично: повторная обработка того же документа всегда даёт те же идентификаторы.

Существенно и устройство ключа хеширования. На вход хеширования подаётся не полное ФИО, а сочетание фамилии и инициалов имени и отчества. Это устраняет реальную проблему документооборота кафедры: один и тот же человек может быть упомянут полным именем в заявлении и инициалами в подписи руководителя. Если бы ключ содержал полные имя и отчество, такие упоминания получили бы разные идентификаторы. Сведение к инициалам объединяет их в одного индивида при крайне маловероятной коллизии в пределах кафедры.

3.2.3 Структурная валидация шаблона через статический анализ

Из шести категорий проверки шаблона (см. 2.4.3) наиболее технически интересны *безопасность* и *согласованность с онтологией*, реализованные через анализ абстрактного синтаксического дерева исходного кода.

Безопасность опирается на обход дерева с проверкой трёх типов узлов: импортов, вызовов и обращений к атрибутам. Списки запрещённых имён зафиксированы в виде констант с пояснением конкретной угрозы.

Каждое замечание помечается уровнем важности и ссылкой на номер строки и колонки в исходном коде шаблона.

Категория согласованности с онтологией выполняет аналогичный обход: извлекаются все обращения к объекту ONTO (программное представление

пространства имён онтологии) — как через атрибут (ONTO.Студент), так и через индекс (ONTO["имеетВКР"]). Имена сверяются с фактическим перечнем терминов схемы. При несовпадении выдаётся предупреждение с подсказкой ближайшего по написанию имени, что облегчает обнаружение опечаток.

3.2.4 Контракт ответа языковой модели

Опыт реализации показал: качество структурированного ответа модели определяется не столько собственно её способностями, сколько детальностью описания контракта в системном промпте. При недостаточной формализации модель регулярно возвращала неоднозначные ответы, не поддающиеся однозначному разбору.

В итоговом виде системный промпт перечисляет пять допустимых сценариев исхода для каждого поля — подтверждение, исправление, отказ при исправлении, нахождение нового значения, отказ при поиске — и накладывает явный инвариант, исключающий комбинации значений, не предусмотренные сценариями (в частности, `status=true` с пустым значением допустим только как «подтверждение уже извлечённого шаблоном»). Разбор ответа на стороне приложения использует сценарную таблицу напрямую: для каждого поля проверяются значения `status` и `value` и применяется ровно тот переход, который предписан системой сценариев; любое отклонение фиксируется как непредвиденная ошибка.

3.2.5 Политики слияния как декларативные аннотации схемы

Политика слияния каждого предиката хранится непосредственно в `TVox` в виде аннотации `:mergePolicy`, что позволяет менять поведение слияния без правки программного кода (рисунок 5):

```

# Идентифицирующее свойство/связь — полная замена
:фамилия rdf:type owl:DatatypeProperty ;
  rdfs:domain :Персона ;
  rdfs:range xsd:string ;
  :mergePolicy :Policy_Set .
:авторВКР rdf:type owl:ObjectProperty ;
  rdfs:domain :ВКР ;
  rdfs:range :Студент ;
  :mergePolicy :Policy_Set .

# Изменяющееся во времени состояние — замена по дате
:обучаетсяВГруппе rdf:type owl:ObjectProperty ;
  rdfs:domain :Студент ;
  rdfs:range :Группа ;
  :mergePolicy :Policy_SetByDate .
:темаВКР rdf:type owl:DatatypeProperty ;
  rdfs:domain :ВКР ;
  rdfs:range xsd:string ;
  :mergePolicy :Policy_SetByDate .

# Множественное свойство — добавление
:телефон rdf:type owl:DatatypeProperty ;
  rdfs:domain :Персона ;
  rdfs:range xsd:string ;
  :mergePolicy :Policy_Add .

```

Рисунок 5 — Фрагмент онтологии, содержащий аннотированные свойства

При запуске репозиторий онтологии один раз обходит схему и формирует словарь «предикат → политика», который затем используется при каждом слиянии. Свойства без явной аннотации получают политику по умолчанию (добавление); появление новых предикатов в онтологии не требует правки модуля интеграции.

3.2.6 Откат документа: история как источник истины для ABox

Источником истины для всего ABox служит *упорядоченная история документов*. Состояние общей модели не хранится отдельно: оно собирается из схемы и истории по запросу, а материализованная модель и журнал событий слияния — производные индексы, при необходимости пересобираемые из истории.

Это позволяет реализовать *откат документа* как тривиальную и безопасную операцию — удалить запись из истории и пересобрать состояние (рисунок 6).


```
def rollback_document(self, document_id: str) -> bool:
    history = self.load_history_entries()
    new_history = [e for e in history if e.document_id != document_id]
    if len(new_history) == len(history):
        return False
    self.save_history_entries(new_history)
    self.rebuild_journal()
    full = self.assemble_full_graph(new_history)
    self.write_combined_ontology(full.graph)
    return True
```

Рисунок 6 — Реализация отката документа из онтологической модели

Аналогично устроена *переобработка*: повторное добавление — замена записи в истории и пересборка. Политики слияния применяются заново, в том числе с участием других документов.

Описанная архитектура даёт четыре свойства одновременно. *Безопасность отката*: операция никогда не оставляет модель в несогласованном состоянии, так как последняя всегда собирается с нуля. *Воспроизводимость состояния*: одна и та же история всегда даёт одну и ту же модель, независимо от прежних операций. *Устойчивость*: при повреждении журнала или материализованной модели они восстанавливаются пересборкой. *Ясная семантика повторной обработки*: «обработать заново» — это всегда «откатить и добавить с тем же идентификатором», независимо от контекста.

3.3 Пользовательский интерфейс

Графический интерфейс реализован средствами PySide6 [24] и организован как единое окно с тремя тематическими вкладками: *Документы*, *Шаблоны*, *Онтология*. Каждая вкладка построена по схеме «список объектов слева, карточка выбранного объекта справа». Ниже описаны существенные особенности каждой вкладки; общий вид — на рисунках 7, 8, 9.

3.3.1 Вкладка документов и редактор графа фактов

Карточка документа состоит из трёх суб-вкладок: *Оригинал* (предпросмотр первой страницы PDF-конверсии и кнопка открытия документа в системной программе), *UDDM* (HTML- и tree-view-режимы с подсветкой структуры) и *Граф* (двухпанельный редактор). Над суб-вкладками — индикатор статуса с

пройденными стадиями и кнопки запуска или продолжения обработки и её сброса с повторным запуском.

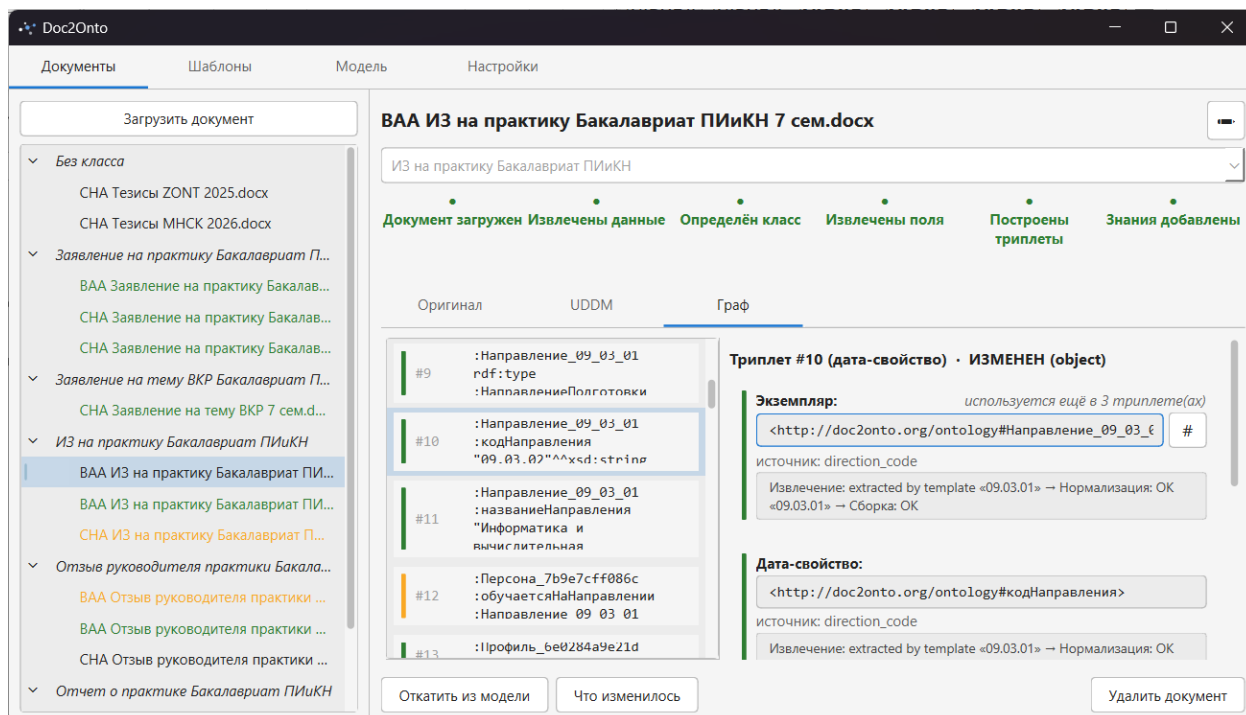


Рисунок 7 — Вкладка документов: суб-вкладка «Граф»

Наибольший функциональный интерес — суб-вкладка *Граф*. Слева отображается список узлов и триплетов: полные триплеты показаны обычным оформлением, неполные выделены цветом, отредактированные пользователем — отдельным оттенком. Справа — форма редактирования: для триплета — управление включением и исключением, для узла — ввод значения и набор вспомогательных операций.

Из этих операций отдельно стоит отметить три, делающих ручную коррекцию удобной даже когда шаблон не построил узел. Для узлов, представляющих индивид класса или типизированный литерал, доступна кнопка *генерации значения через концепт*: пользователь выбирает концепт, вводит значение в человекочитаемой форме, и интерфейс прозрачно выполняет полный цикл — разбор, нормализацию, вычисление IRI с автоматическим формированием идентифицирующих литералов или типизированного литерала; перед подтверждением показывается *предпросмотр итогового IRI или литерала*. Для каждого ранее отредактированного узла рядом с формой появляется кнопка

сброса к исходному значению, по которой переопределение из слоя правок удаляется. Совокупность этих средств делает редактирование графа и диагностику ошибок возможными без выхода в исходный код шаблона.

3.3.2 Вкладка шаблонов

Карточка шаблона содержит описание, исходный код с подсветкой синтаксиса (через Pygments с преобразованием в HTML), сводный отчет о структурной валидации и кнопки операций.

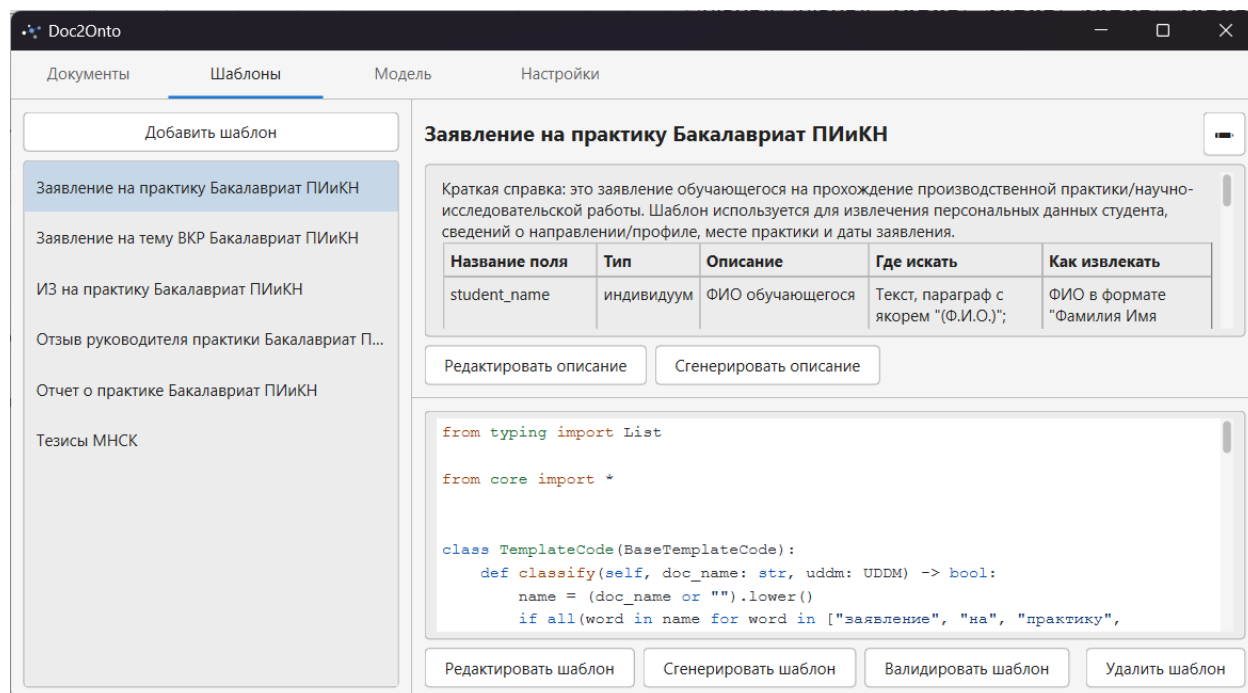


Рисунок 8 — Вкладка шаблонов: карточка шаблона с кодом и описанием

Кнопка *генерации шаблона* запускает многошаговый сценарий автоматического создания шаблона на основе образца документа и схемы онтологии: первое обращение к языковой модели формирует описание класса документа и таблицу полей; второе обращение — по этому описанию — генерирует исполняемый код шаблона. Окончательная доводка возлагается на автора; перед использованием шаблон проходит структурную валидацию. Кнопка *открытия в редакторе* запускает внешнюю программу редактирования: встроенный редактор не реализован, поскольку для написания кода логично использовать полноценную среду с привычной автору конфигурацией.

3.3.3 Вкладка онтологии и навигация по модели

Левая часть содержит иерархию классов и список индивидов; правая — карточку выбранного класса или индивида.

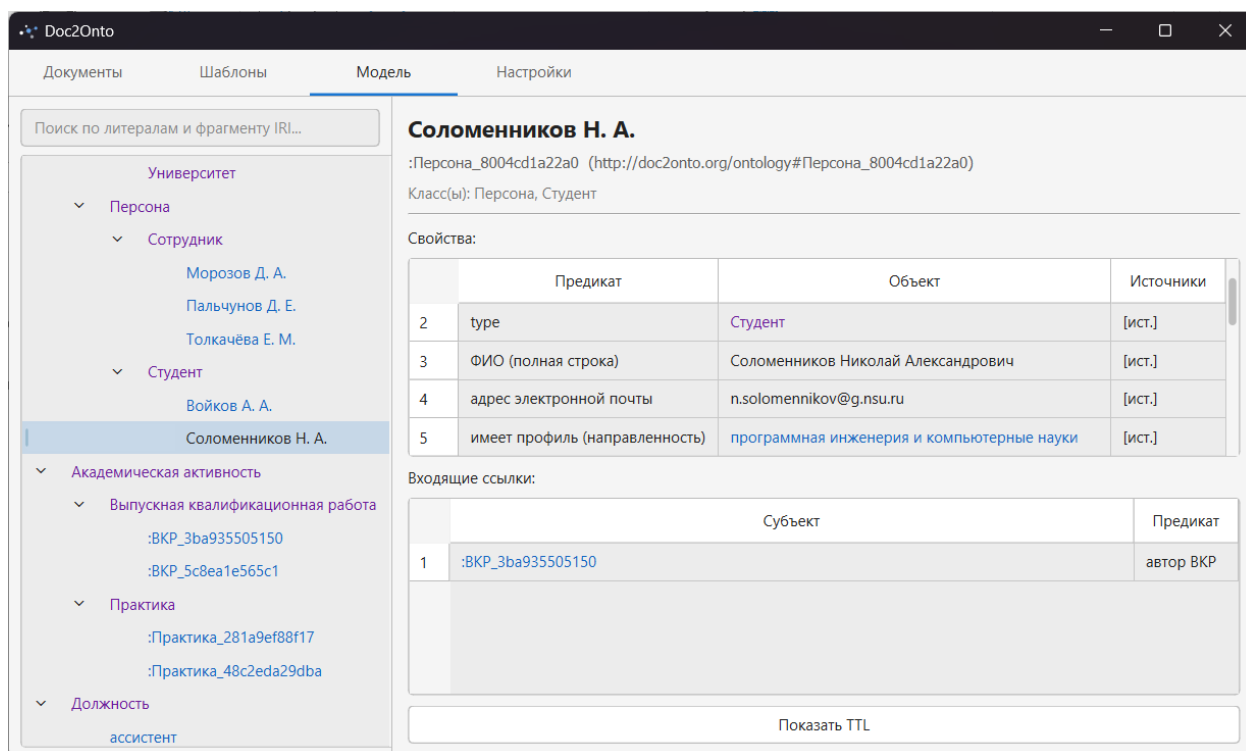


Рисунок 9 — Вкладка онтологии: карточка индивида с переходами между связанными сущностями

Особенность реализации — *внутренняя навигация между связанными сущностями*. Каждое имя класса или индивида в карточке отображается как гиперссылка; по щелчку левая часть переключается на соответствующий объект, а правая открывает его карточку. Это позволяет переходить, например, от карточки студента — к его ВКР по связи *автор ВКР*, далее — к научному руководителю, далее — к организации, не покидая вкладки и не прибегая к глобальному поиску. Поддерживаются переходы и от индивида к его типу-классу, и от класса к перечню его индивидов.

Для каждого факта в карточке доступен диалог *источника факта* с указанием документа, шаблона, эффективной даты, применённой политики слияния и сведений о вытесненных конкурирующих фактах. Из диалога возможен прямой переход к карточке документа-источника. В сочетании с

внутренней навигацией это обеспечивает полную прослеживаемость каждого факта от его места в общей модели до фрагмента исходного документа.

3.3.4 Известные ограничения интерфейса

Интерфейс реализован в синхронной модели обработки событий: длительные операции (обращение к языковой модели, многошаговая генерация шаблона, прогон конвейера на крупном документе) приводят к временной невосприимчивости главного окна. Это известное ограничение, устраняемое переводом длительных операций в рабочие потоки или асинхронные задачи без изменения остальных частей системы.

3.4 Апробация системы

3.4.1 Тестовый набор документов и проверенные сценарии

Качественная апробация проводилась на наборе реальных документов кафедры: заявлении обучающегося о теме ВКР, двух заявлениях на практику за разные семестры, индивидуальном задании на практику, отчёте руководителя практики, тезисах двух студенческих научных конференций. Все документы относятся к одному обучающемуся, что обеспечивает наличие в полученной модели согласованных перекрёстных ссылок на одного индивида. Имена обезличены.

В рамках апробации проверены четыре сценария.

Полный цикл обработки документа. Каждый документ проведён через все пять стадий конвейера. Для всех документов конвейер достиг последней стадии без ручного вмешательства; графы фактов оказались полными; интеграция в общую модель прошла без нарушений целостности. Полученная модель содержит согласованные сведения о студенте, его ВКР, прохождении практик в двух семестрах, научных публикациях, направлении подготовки и профиле, учебной группе, профильных организациях прохождения практики и руководителях.

Кросс-форматный тест. Один из документов обработан в трёх форматах: исходный DOCX, преобразованный к DOC, преобразованный к PDF. Унифицированные представления оказались идентичны для пары DOCX/DOC и

близки до уровня переноса абзацев для PDF; результаты извлечения полей и итоговые графы фактов совпали с точностью до канонической формы. Результат подтверждает свойство форматной независимости правил извлечения, обоснованное в главе 2.

Структурно близкие документы под одним шаблоном. Два заявления на практику за разные семестры обрабатывались одним шаблоном. В первом случае декларативная часть справилась полностью; во втором ряд полей был «вытянут» только этапом коррекции из-за локального сдвига структуры. Графы фактов в обоих случаях оказались полными. Сценарий иллюстрирует ценность гибридной экстракции: коррекция языковой моделью обработала документ, для которого правила оказались неточны, без правки шаблона.

Ручное редактирование графа. На примере одного документа в графическом интерфейсе выполнено создание дополнительного индивида через концепт, добавление триплета связи, исключение ложноположительного триплета, переопределение значения литерала. Правки сохранены и при повторном прогоне стадии сборки графа автоматически применены к новому черновику.

3.4.2 Качественная оценка результатов

По итогам апробации можно сделать следующие выводы. Полный цикл обработки документа проходит без ручного вмешательства в типичных случаях; все функциональные требования главы 1 реализованы. Воспроизводимость детерминированной части подтверждена многократными прогонами; объяснимость обеспечена сохранением полной истории извлечения и доступностью её просмотра в графическом интерфейсе. Расширяемость подтверждена опытом разработки шаблонов для разных классов документов: подключение нового типа выполняется добавлением шаблона.

Опыт эксплуатации подтвердил основной тезис главы 1: гибридный подход «детерминированные правила + коррекция языковой моделью» сочетает воспроизводимость и устойчивость к вариативности. Случаи неточности правил

успешно компенсируются коррекцией модели; основная масса значений извлекается детерминированной частью, обеспечивая прозрачность процесса.

Среди особенностей, замеченных в ходе апробации: качество автоматически сгенерированной заготовки шаблона существенно зависит от похожести нового класса на ранее виденные; при необычных формулировках дат разбор эффективной даты может быть неудачным, что препятствует корректному применению политики «замены по дате»; некоторые ошибки в логике связывания фактов в шаблоне обнаруживаются только в реальных запусках и не выявляются структурной валидацией.

Сознательно *не* проверялись и не должны выдаваться за достижения: количественные метрики качества (в отсутствие размеченного корпуса); работа на сканированных документах; многопользовательский режим; обработка очень крупных документов (сотни страниц); устойчивость генерации шаблонов для классов, существенно отличающихся от ранее обработанных моделью.

3.5 Известные ограничения и направления развития

По результатам разработки и апробации определены направления дальнейшего развития. Большинство ограничений — сознательно принятые компромиссы, допускающие устранение без полной переработки системы благодаря модульному устройству архитектуры.

- **Поддержка сканированных документов.** Архитектура содержит каркас для подключения внешних извлекателей данных (см. 2.3.2); интеграция с одной из систем оптического распознавания символов реализуется отдельным классом без изменения остальной части.

- **Песочница исполнения шаблонов.** Текущая реализация обеспечивает безопасность только статическим анализом. Полноценная песочница (специализированный интерпретатор, изолированный подпроцесс, контейнеризация) расширила бы круг возможных авторов шаблонов.

- **Многопользовательский режим.** Одновременный доступ нескольких пользователей к общей модели потребует механизмов блокировок или транзакционного слоя поверх репозитория онтологии.

- **Замена локального хранения на специализированное хранилище RDF-триплетов.** Менеджеры хранилища изолируют остальные части системы от способа хранения; при росте объёма данных переход выполняется заменой реализации менеджеров.

- **Поддержка локально развёрнутых языковых моделей.** Зависимость от внешнего сервиса устраняется переходом к моделям, доступным через совместимые программные интерфейсы.

- **Введение машины логического вывода.** Применение OWL-reasoner-а расширит возможности контроля целостности модели.

- **Расширенная апробация.** Формирование размеченного корпуса с эталонными значениями полей и расчёт количественных метрик качества — самостоятельное направление исследовательской работы.

Принятая архитектура с разделением на функционально независимые модули обеспечивает последовательное устранение каждого из ограничений без необходимости полной переработки системы.

Подробное описание программы — в приложении Г, а руководство оператора — в приложении Д.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработана настольная программная система Doc2Onto для автоматизированного пополнения онтологической модели кафедры знаниями, извлечёнными из документов учебно-организационной деятельности.

В процессе работы решены следующие задачи.

1. Проанализированы 4 категории решений: Apache Tika, GROBID, облачные IDP-сервисы Google / Azure / Amazon, исследовательские системы Karma и KnowledgeStore; обоснована необходимость собственной разработки.
2. Сформулированы 11 функциональных и 6 нефункциональных требований, приняты 4 проектных ограничения.
3. Введена унифицированная модель представления документов.
4. Разработан конвейер обработки документа из 5 стадий.
5. Разработан механизм шаблонов как Python-модулей со структурной валидацией по 6 категориям, включая AST-анализ безопасности.
7. Разработан декларативный язык извлечения значений полей и реализована гибридная экстракция «детерминированный шаблон + коррекция большой языковой моделью».
8. Реализована концептно-ориентированная модель идентификации индивидов: 14 концептов, 4 стратегии формирования идентификаторов; для ФИО — морфологическое приведение к именительному падежу через rymorphy3.
9. Реализованы 3 политики слияния значений предикатов, заданные декларативно в схеме онтологии аннотацией `:mergePolicy`.
10. Реализован откат документа из онтологической модели.
11. Реализован графический интерфейс на PySide6 с тремя тематическими вкладками и сквозной прослеживаемостью каждого факта до фрагмента исходного документа.
12. Реализована подсистема взаимодействия с LLM (OpenAI SDK).

Программная часть — более 12 тысяч строк собственного исходного кода на языке Python (без учёта пустых строк и комментариев). Качественная

апробация проведена на 15 реальных документах кафедры; для всех документов конвейер достиг последней стадии без ручного вмешательства, кросс-форматный тест DOCX / DOC / PDF одного документа дал идентичный итоговый граф.

Работа представлена автором на 64-й Международной научной студенческой конференции (Новосибирск, 15–21 апреля 2026 г.) [27], по итогам которой работа отмечена дипломом III степени. Программная система согласована с секретарём кафедры общей информатики.

Перспективы развития: интеграция средств оптического распознавания символов через предусмотренный архитектурный слот; полноценная песочница исполнения шаблонов; многопользовательский режим со специализированным хранилищем RDF-триплетов; поддержка локально развёрнутых языковых моделей; применение OWL-reasoner-a; формирование размеченного корпуса для расчёта количественных метрик качества извлечения.

Выпускная квалификационная работа выполнена мной самостоятельно и с соблюдением правил профессиональной этики. Все использованные в работе материалы и заимствованные принципиальные положения (концепции) из опубликованной научной литературы и других источников имеют ссылки на них. Я несу ответственность за приведенные данные и сделанные выводы.

Я ознакомлен с программой государственной итоговой аттестации, согласно которой обнаружение плагиата, фальсификации данных и ложного цитирования является основанием для не допуска к защите выпускной квалификационной работы и выставления оценки «неудовлетворительно».

Соломенников Николай Александрович

ФИО студента

Подпись студента

« ____ » _____ 20 ____ г.

(заполняется от руки)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» // Собрание законодательства РФ. — 2006. — № 31 (ч. I). — Ст. 3451.
2. ISO/IEC 29500-1:2016. Information technology — Document description and processing languages — Office Open XML File Formats — Part 1: Fundamentals and Markup Language Reference. — Geneva : ISO, 2016.
3. ISO 32000-1:2008. Document management — Portable document format — Part 1: PDF 1.7. — Geneva : ISO, 2008.
4. Gruber T. R. A translation approach to portable ontology specifications // Knowledge Acquisition. — 1993. — Vol. 5, № 2. — P. 199–220.
5. The Description Logic Handbook: Theory, Implementation, and Applications / F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, P. F. Patel-Schneider (Eds.). — 2nd ed. — Cambridge : Cambridge University Press, 2007. — 622 p.
6. Maedche A., Staab S. Ontology Learning for the Semantic Web // IEEE Intelligent Systems. — 2001. — Vol. 16, № 2. — P. 72–79.
7. Cimiano P. Ontology Learning and Population from Text : Algorithms, Evaluation and Applications. — New York : Springer, 2006. — 347 p.
8. Nadeau D., Sekine S. A survey of named entity recognition and classification // Lingvisticae Investigationes. — 2007. — Vol. 30, № 1. — P. 3–26.
9. Brown T. B., Mann B., Ryder N. et al. Language Models are Few-Shot Learners // Advances in Neural Information Processing Systems 33 (NeurIPS 2020). — 2020. — P. 1877–1901.
10. Lopez P. GROBID: Combining Automatic Bibliographic Data Recognition and Term Extraction for Scholarship Publications // Proceedings of the 13th European Conference on Research and Advanced Technology for Digital Libraries (ECDL 2009). — Berlin, Heidelberg : Springer, 2009. — P. 473–474.
11. Knoblock C. A., Szekely P., Ambite J. L. et al. Semi-automatically Mapping Structured Sources into the Semantic Web // Proceedings of the 9th Extended

Semantic Web Conference (ESWC 2012). — LNCS, vol. 7295. — Berlin, Heidelberg : Springer, 2012. — P. 375–390.

12. Corcoglioniti F., Rospoche M., Cattoni R., Magnini B., Serafini L. Interlinking Unstructured and Structured Knowledge in an Integrated Framework // Proceedings of the 7th IEEE International Conference on Semantic Computing (ICSC 2013). — Irvine, CA : IEEE, 2013. — P. 40–47.

13. Korobov M. Morphological Analyzer and Generator for Russian and Ukrainian Languages // Analysis of Images, Social Networks and Texts (AIST 2015). — Communications in Computer and Information Science, vol. 542. — Cham : Springer, 2015. — P. 320–332.

14. Mattmann C. A., Zitting J. L. Tika in Action. — Greenwich, CT : Manning Publications, 2011. — 256 p.

15. OWL 2 Web Ontology Language Document Overview (Second Edition) : W3C Recommendation 11 December 2012 [Электронный ресурс]. — URL: <https://www.w3.org/TR/owl2-overview/>

16. RDF 1.1 Concepts and Abstract Syntax : W3C Recommendation 25 February 2014 / R. Cyganiak, D. Wood, M. Lanthaler [Электронный ресурс]. — URL: <https://www.w3.org/TR/rdf11-concepts/>

17. Apache Tika — a content analysis toolkit [Электронный ресурс]. — URL: <https://tika.apache.org/>

18. GROBID Documentation [Электронный ресурс]. — URL: <https://grobid.readthedocs.io/>

19. Karma : A Data Integration Tool [Электронный ресурс]. — URL: <https://usc-isi-i2.github.io/karma/>

20. Google Cloud Document AI [Электронный ресурс]. — URL: <https://cloud.google.com/document-ai>

21. Microsoft Azure AI Document Intelligence [Электронный ресурс]. — URL: <https://azure.microsoft.com/en-us/products/ai-services/ai-document-intelligence>

22. Amazon Textract [Электронный ресурс]. — URL: <https://aws.amazon.com/textract/>
23. The Python Language Reference, version 3.11 [Электронный ресурс]. — URL: <https://docs.python.org/3.11/reference/>
24. Qt for Python (PySide6) Documentation [Электронный ресурс]. — URL: <https://doc.qt.io/qtforpython-6/>
25. RDFLib Documentation [Электронный ресурс]. — URL: <https://rdflib.readthedocs.io/>
26. OpenAI Python Library [Электронный ресурс]. — URL: <https://github.com/openai/openai-python>
27. Соломенников Н. А. Разработка системы автоматизированного пополнения онтологической модели кафедры знаниями, извлечёнными из документов // Материалы 64-й Международной научной студенческой конференции. — Новосибирск, 15–21 апреля 2026 г.

ПРИЛОЖЕНИЕ А

Ниже приведён сокращённый пример унифицированного представления документа — заявления обучающегося на прохождение практики (рисунок А.1). Класс документа описан шаблоном в Приложении В. Пример иллюстрирует использование всех трёх типов блоков (текст, список, таблица), вложенных абзацев и табличных данных.

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
  <text>
    <p>Министерство науки и высшего образования Российской Федерации</p>
    <p>Новосибирский национальный исследовательский государственный университет</p>
    <p>Факультет информационных технологий</p>
    <p>Кафедра общей информатики</p>
  </text>
  <text>
    <p>ЗАЯВЛЕНИЕ</p>
    <p>о направлении на практику</p>
  </text>
  <text>
    <p>От обучающегося Соломенникова Николая Александровича (Ф.И.О.)</p>
    <p>4 курса, группы № 22204</p>
    <p>направление 09.03.01 (код и наименование направления)</p>
    <p>Информатика и вычислительная техника</p>
    <p>направленность (профиль) Программная инженерия и компьютерные науки (наименование профиля)</p>
  </text>
  <text>
    <p>Прошу направить меня на производственную практику</p>
    <p>(указывается наименование практики)</p>
    <p>в ФГБУН Институт математики им. С. Л. Соболева СО РАН</p>
    <p>630090, г. Новосибирск, пр. Академика Коптюга, 4</p>
  </text>
  <table>
    <row>
      <cell><text><p>Дата начала практики</p></text></cell>
      <cell><text><p>29.09.2025</p></text></cell>
    </row>
    <row>
      <cell><text><p>Дата окончания практики</p></text></cell>
      <cell><text><p>23.12.2025</p></text></cell>
    </row>
  </table>
  <text>
    <p>Дата: «20» сентября 2025 г.</p>
  </text>
  <text>
    <p>Руководитель ВКР Пальчунов Дмитрий Евгеньевич, зав. кафедрой</p>
    <p>(Ф.И.О. полностью) (должность)</p>
  </text>
</root>
```

Рисунок А.1 — Краткий пример документа в унифицированном формате

ПРИЛОЖЕНИЕ Б

Ниже приведена схема онтологии кафедры, иллюстрирующая классы и свойства, связывающий индивиды их данных классов (рисунок Б.1). Для аннотации `:mergePolicy` указаны экземпляры (индивиды). Полный текст схемы доступен в исходных материалах системы.

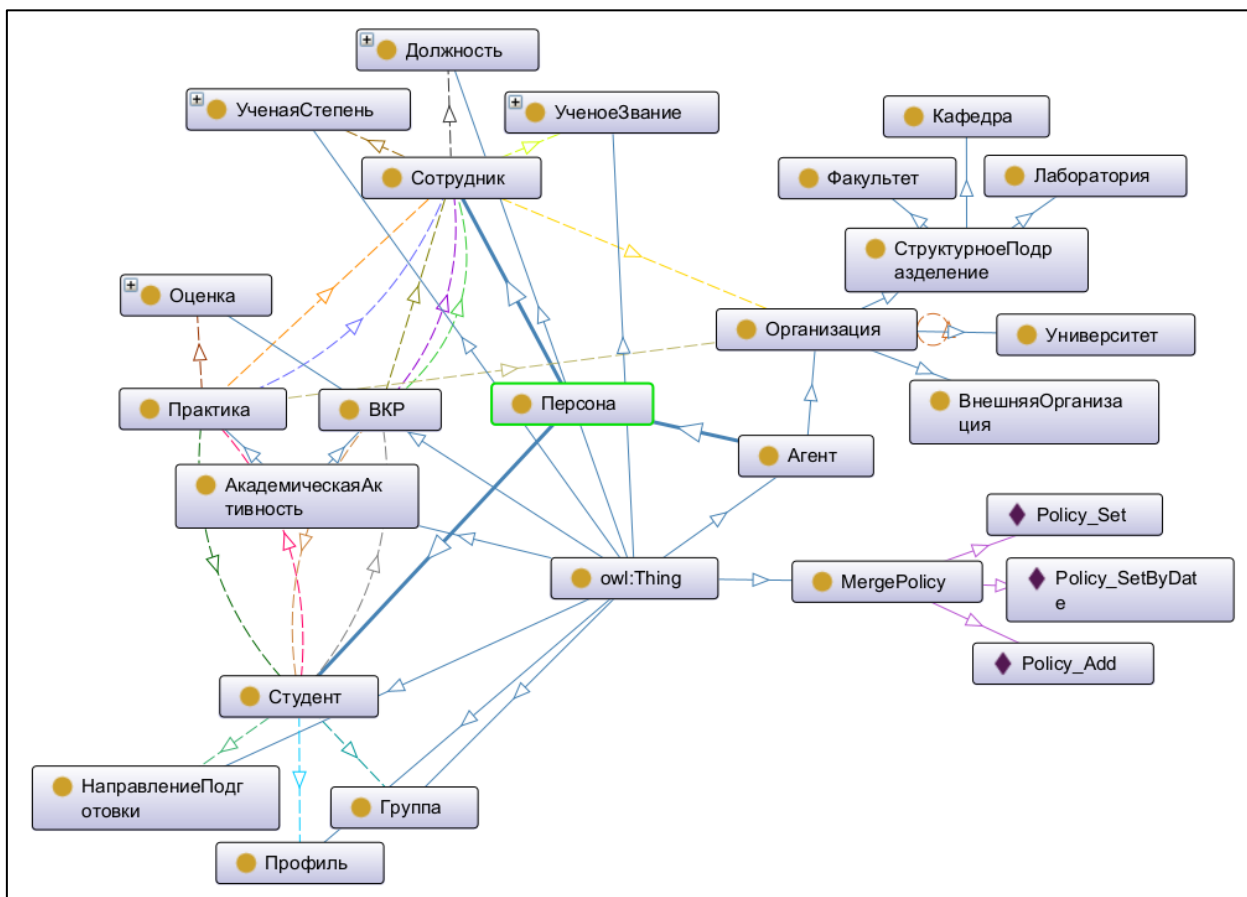


Рисунок Б.1 — Схема онтологии кафедры

ПРИЛОЖЕНИЕ В

Ниже приведён сокращённый пример кода шаблона для класса документа «заявление обучающегося на прохождение практики» (рисунок В.1). Шаблон включает все три обязательных метода контракта: правила распознавания принадлежности документа классу (`classify`), описание извлекаемых полей (`fields`) и правила сборки графа фактов (`build`).

```
from typing import List

from core import *

class TemplateCode(BaseTemplateCode):

    def classify(self, doc_name: str, uddm: UDDM) -> bool:
        name = (doc_name or "").lower()
        if all(w in name for w in ["заявление", "практику", "пиикн"]):
            return True

        full_text = str(uddm.root).lower()
        return all(w in full_text for w in [
            "заявление", "прошу направить меня", "практику",
        ])

    def fields(self) -> List[Field]:
        return [
            Field(
                "student_name", "ФИО обучающегося",
                sel().find(ElementType.P, Predicate.contains_text("(Ф.И.О.)")),
                ext().before("(Ф.И.О.)").keep_letters_and_spaces().normalize_spaces(),
                norm().concept(PersonConcept),
            ),
            Field(
                "course_number", "Номер курса обучения",
                sel().find(ElementType.P, Predicate.contains_text("курса")),
                ext().regex(r"(\d+)\s*курса", group=1),
                norm().integer().in_range(1, 4),
            ),
            # << другие поля... >>
        ]

    def build(self, b: TemplateGraphBuilder):
        student = b.individual("student_name", PersonConcept, role=ONTO.Студент)

        b.add_object_property(student, ONTO.обучаетсяВГруппе,
                               b.individual("group_number", GroupConcept))
        b.add_object_property(student, ONTO.обучаетсяНаНаправлении,
                               b.direction("direction_code", name_field="direction_name"))
        b.add_object_property(student, ONTO.имеетПрофиль,
                               b.individual("profile_name", ProfileConcept))

        b.individual("organization_full_name", OrganizationConcept, role=ONTO.ВнешняяОрганизация)

        supervisor = b.individual("supervisor_name", PersonConcept, role=ONTO.Сотрудник)
        b.add_object_property_optional(supervisor, ONTO.занимаетДолжность,
                                       b.individual("supervisor_position", PositionConcept))

        thesis = b.thesis(student=student)
        b.add_object_property(thesis, ONTO.руководительВКР, supervisor)
        b.add_object_property(student, ONTO.имеетВКР, thesis)
```

Рисунок В.1 — Пример кода шаблона извлечения знаний

ПРИЛОЖЕНИЕ Г

ПРОГРАММА «Doc2Onto» ОПИСАНИЕ ПРОГРАММЫ

Листов 13

Новосибирск, 2026

СОДЕРЖАНИЕ

Аннотация	67
1 Общие сведения.....	68
1.1 Обозначение и наименование программы	68
1.2 Программное обеспечение, необходимое для функционирования программы	68
1.3 Языки программирования	68
2 Функциональное назначение	69
3 Описание логической структуры.....	71
3.1 Архитектура программы	71
3.2 Структура программы	71
4 Используемые технические средства	73
5 Вызов и загрузка.....	74
6 Входные данные	75
7 Выходные данные.....	76
8 Лист регистрации изменений.....	77

АННОТАЦИЯ

В настоящем документе приведено описание программной системы Doc2Onto — настольного приложения для автоматизированного пополнения онтологической модели кафедры знаниями, извлечёнными из документов учебно-организационной деятельности. Система реализует полный цикл обработки документа — от загрузки исходного файла до записи извлечённых фактов в общую онтологическую модель — на основе гибридного подхода «детерминированный шаблон + коррекция большой языковой моделью». Документ оформлен в соответствии с требованиями ЕСПД (ГОСТ 19.402-78, ГОСТ 19.105-78).

1 Общие сведения

1.1 Обозначение и наименование программы

Наименование программы — **Doc2Onto**.

1.2 Программное обеспечение, необходимое для функционирования программы

Система реализована как настольное приложение с единым процессом исполнения и функционирует в следующем окружении:

- интерпретатор языка Python версии 3.11;
- кроссплатформенная библиотека графического интерфейса PySide6 (Qt 6);
- библиотека работы с RDF-графами rdflib;
- библиотека морфологической обработки русского языка rymorphy3;
- официальный программный интерфейс OpenAI Python (используется для извлечения значений полей и автоматизированной генерации шаблонов; модель gpt-5.4-mini по умолчанию);
- внешний пакет LibreOffice либо Microsoft Word с компонентом COM-доступа (через библиотеку pywin32) — для преобразования документов формата DOC к формату DOCX и предпросмотра PDF-копий в графическом интерфейсе;
- внешний веб-сервис ConvertAPI — для преобразования документов формата PDF к формату DOCX.

Кроссплатформенность обеспечена за счёт использования Python и Qt: программа работает на операционных системах семейства Microsoft Windows и в основных дистрибутивах Linux.

1.3 Языки программирования

Программа реализована на языке программирования общего назначения Python (версия 3.11) — как кодовая база приложения, так и пользовательские шаблоны извлечения знаний.

2 Функциональное назначение

Программа предназначена для решения следующих задач:

- автоматизированное преобразование документов кафедры различных форматов (DOC, DOCX, PDF с текстовым слоем) к унифицированному иерархическому представлению, изолирующему форматные особенности от логики извлечения знаний;
- определение класса документа путём перебора имеющихся шаблонов с возможностью ручного указания класса при отказе автоматической классификации;
- извлечение значений объявленных шаблоном полей в гибридном режиме: выполнение детерминированных правил шаблона с последующей проверкой и коррекцией значений большой языковой моделью;
- приведение извлечённых значений к канонической форме с применением концептов онтологии (морфологическое приведение ФИО к именительному падежу, нормализация дат, сопоставление текстовых форм с именованными индивидами перечислений);
- построение чернового графа фактов в терминах онтологической модели с сохранением метаданных о неполных узлах и их источниках;
- предоставление пользователю возможности ручного редактирования графа с двухслойным сохранением правок (черновик от шаблона отдельно, правки пользователя отдельно);
- интеграция извлечённых фактов в общую онтологическую модель кафедры с учётом политик слияния, заданных в схеме онтологии, и эффективной даты документа;
- автоматизированная генерация заготовок шаблонов на основе образцов документов и схемы онтологии;
- структурная валидация пользовательских шаблонов с AST-анализом исходного кода;

- отслеживание происхождения каждого факта в общей модели до исходного документа, шаблона и применённой политики слияния;
- откат документа — удаление всех ассоциированных с ним фактов с безопасной пересборкой общей модели.

3 Описание логической структуры

3.1 Архитектура программы

Программа разделена на шесть функциональных слоёв (схема архитектуры приведена на рисунке 2.6 основного текста работы):

1. **Слой описаний данных** — структуры передачи данных между остальными слоями (документ, шаблон, результат извлечения полей, черновой граф, слой пользовательских правок).

2. **Слой ядра** — переиспользуемые средства, доступные коду пользовательских шаблонов: модель унифицированного представления документов, декларативный язык извлечения значений полей (поле, селектор, извлекатель, нормализатор), концепты онтологии, построитель графа фактов, контракт шаблона и средства его структурной валидации.

3. **Слой модулей конвейера** — реализации пяти стадий обработки документа: преобразование к унифицированному представлению, определение класса документа, извлечение значений полей, построение чернового графа фактов, интеграция в общую онтологическую модель.

4. **Слой работы с файлами** — менеджеры документов и шаблонов, репозиторий онтологии; единственная точка контакта с файловой системой.

5. **Слой оркестрации** — общий контекст приложения с глобальными сервисами, конвейер обработки, подсистема обращения к большой языковой модели, настройки, журналирование.

6. **Слой пользовательского интерфейса** — графический интерфейс на основе PySide6 с тремя тематическими вкладками для работы с документами, шаблонами и онтологической моделью.

3.2 Структура программы

Обработка одного документа реализована конвейером из пяти последовательных стадий, каждая из которых принимает результат предыдущей и переводит документ из одного состояния в следующее. Состояние документа явно фиксируется после каждой стадии в персистентном хранилище, что обеспечивает возобновляемость работы.

Хранилище онтологической модели организовано трёхслойно: фиксированная read-only схема онтологии (TBox), упорядоченная история добавленных документов (источник истины для всей наполняющей части модели — ABox) и журнал событий слияния (производный append-only-индекс). Материализованное представление полной модели пересобирается лениво из схемы и истории с кэшированием по отпечатку истории.

Шаблоны извлечения знаний являются программными модулями на языке Python с фиксированным интерфейсом и хранятся в отдельных подкаталогах. Контракт шаблона включает три обязательных метода: правила распознавания принадлежности документа классу, описание извлекаемых полей и правила сборки графа фактов. Безопасность исполнения пользовательских шаблонов обеспечивается структурной валидацией с AST-анализом исходного кода: проверкой отсутствия импортов опасных модулей, обращений к опасным встроенным функциям и магическим атрибутам интерпретатора, корректности списка полей, согласованности обращений к терминам онтологии со схемой.

4 Используемые технические средства

- **ОС:** Microsoft Windows 10/11 либо современный дистрибутив Linux (Ubuntu 22.04 LTS и аналогичные);
- **Аппаратные требования:** процессор класса не ниже Intel Core i5 / AMD Ryzen 5, оперативная память не менее 4 ГБ (рекомендуется 8 ГБ — связано с загрузкой словарей rymorphy3), дисковое пространство для рабочих данных от 500 МБ;
- **Программное обеспечение:** интерпретатор Python 3.11; LibreOffice (или Microsoft Word) для конвертации документов формата DOC; доступ в сеть Интернет для обращения к программному интерфейсу OpenAI и веб-сервису ConvertAPI;
- **Среда разработки:** Visual Studio Code или JetBrains PyCharm с настроенным интерпретатором Python и встроенной проверкой типов (например, через расширение Pylance);
- **Библиотеки:** PySide6 6.x, rdflib 7.x, rymorphy3 1.x, OpenAI Python SDK, pydantic, Pygments, Markdown2.

5 Вызов и загрузка

Подготовка окружения и запуск программы выполняются в следующей последовательности.

1. Создание виртуального окружения Python: `py -3.11 -m venv .venv`.
2. Активация окружения: `.venv\Scripts\activate` (Windows) либо `source .venv/bin/activate` (Linux).
3. Обновление инструментов сборки: `python -m pip install --upgrade pip setuptools wheel`.
4. Установка зависимостей: `pip install -r requirements.txt`.
5. Создание файла переменных окружения `.env` из шаблона `.env.example` со значениями ключа `OPENAI_API_KEY` для доступа к OpenAI и ключа `CONVERTAPI_SECRET` для доступа к сервису преобразования PDF.
6. Установка LibreOffice (<https://www.libreoffice.org/>) либо проверка наличия установленного Microsoft Word на компьютере пользователя.
7. Запуск приложения: `python main.py`.

При первом запуске программа создаёт каталог рабочих данных `.data/` с подкаталогами для документов, шаблонов, онтологии и журналов; в каталог онтологии автоматически копируется поставляемая схема `resources/onto/schema.ttl`. Все последующие запуски используют уже сформированное состояние.

6 Входные данные

Программа принимает на вход:

- **исходные документы** в форматах DOC, DOCX, PDF с текстовым слоем; загружаются пользователем через графический интерфейс (вкладка «Документы»);
- **шаблоны извлечения знаний** в виде программных модулей на языке Python с реализацией контракта BaseTemplateCode; добавляются вручную через интерфейс или создаются с помощью автоматизированной генерации на основе образцов документов;
- **схема онтологии кафедры** в формате Turtle (RDF) — поставляется вместе с приложением как read-only артефакт;
- **переменные окружения**: ключ доступа к программному интерфейсу OpenAI и ключ доступа к веб-сервису ConvertAPI;
- **дополнительные факты** — самостоятельные TTL-файлы, которые пользователь может добавить к конкретному документу для пополнения графа фактами, не покрытыми шаблоном.

7 Выходные данные

Программа формирует следующие выходные артефакты:

- **общая онтологическая модель кафедры** в формате Turtle, пересобираемая по истории документов и содержащая факты из всех обработанных документов в виде RDF-графа в терминах схемы онтологии;
- **унифицированное представление каждого документа** в формате XML и его вспомогательные визуализации (плоский текст, HTML, текстовое дерево);
- **результат извлечения значений полей** для каждого документа — с историей всех трёх этапов (детерминированный, языковая модель, нормализация);
- **черновой граф фактов** для каждого документа с метаданными о неполных узлах и их источниках;
- **слой пользовательских правок** чернового графа в отдельном файле;
- **итоговый граф документа** (после применения правок и дополнительных фактов) в формате Turtle;
- **журнал событий слияния** (facts.jsonl) с полным контекстом каждого события (добавление, отзыв, отклонение факта);
- **отчёт о слиянии** документа с указанием произведённых изменений в общей модели;
- **журнал работы приложения** (app.log);
- **журнал обращений к языковой модели** (agents.log) с полными записями системного и пользовательского промптов, ответа модели и длительности обращения.

8 Лист регистрации изменений

Лист регистрации изменений приведён в таблице Г.1.

Таблица Г.1 — Лист регистрации изменений в программном документе «Описание программы».

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц) в документе	№ документа	Входящий № сопроводительного документа	Подпись	Дата
Номер Изм.	измененных	замененных	новых	аннулированных					

ПРИЛОЖЕНИЕ Д

ПРОГРАММА «Doc2Onto» РУКОВОДСТВО ОПЕРАТОРА

Листов 15

Новосибирск, 2026

СОДЕРЖАНИЕ

Аннотация	80
1 Назначение программы.....	81
1.1 Функциональное назначение программы	81
1.2 Эксплуатационное назначение программы	81
1.3 Состав функций	81
2 Условия выполнения программы.....	83
2.1 Минимальный состав аппаратных и программных средств	83
2.2 Требования к оператору	83
3 Выполнение программы	85
3.1 Загрузка и запуск программы	85
3.2 Типовой сценарий обработки документа	85
3.3 Ручное указание класса документа	86
3.4 Ручное редактирование чернового графа фактов	87
3.5 Добавление дополнительных фактов вне шаблона	87
3.6 Создание и редактирование шаблонов извлечения	88
3.7 Просмотр онтологической модели	88
3.8 Откат и переобработка документа	89
3.9 Завершение программы	89
4 Сообщения оператору	90
5 Лист регистрации изменений.....	92

АННОТАЦИЯ

В настоящем документе приведено руководство пользователя по работе с программной системой Doc2Onto — настольным приложением для автоматизированного пополнения онтологической модели кафедры знаниями, извлекаемыми из документов учебно-организационной деятельности. Документ описывает развёртывание системы, последовательность действий оператора при типовых сценариях эксплуатации (загрузка и обработка документа, создание и редактирование шаблона извлечения, ручное редактирование чернового графа, работа с онтологической моделью, откат и переобработка документов) и виды диагностических сообщений системы. Документ оформлен в соответствии с требованиями ЕСПД (ГОСТ 19.505-79, ГОСТ 19.105-78).

1 Назначение программы

1.1 Функциональное назначение программы

Программа предназначена для автоматизированного извлечения знаний из документов учебно-организационной деятельности кафедры и их последующей интеграции в единую онтологическую модель. Обработка выполняется по конвейерной схеме, включающей преобразование документа к унифицированному представлению, автоматическое определение его класса, гибридное извлечение значений полей (детерминированные правила шаблона с последующей проверкой и коррекцией большой языковой моделью), построение чернового графа фактов в терминах онтологии и интеграцию извлечённых фактов в общую модель с учётом политик слияния.

1.2 Эксплуатационное назначение программы

Программный продукт ориентирован на сотрудников кафедры, ответственных за ведение онтологической модели и работу с документами учебного процесса. Программа позволяет пользователю самостоятельно выполнять полный цикл обработки документа без обращения к разработчикам, контролируя и при необходимости корректируя результат на каждом этапе.

1.3 Состав функций

Программа предоставляет следующие функциональные возможности:

1. загрузка и каталогизация исходных документов кафедры в поддерживаемых форматах (DOC, DOCX, PDF с текстовым слоем);
2. просмотр документа в трёх представлениях — оригинального файла, унифицированного иерархического представления и графа фактов;
3. автоматическая обработка документа по полному конвейеру или по отдельным стадиям с возможностью возобновления прерванной обработки;
4. явное указание класса документа при отказе автоматической классификации;
5. ручное редактирование чернового графа фактов: исключение ложноположительных триплетов, переопределение значений узлов, добавление

новых индивидов через концепты с предпросмотром результата, сброс отдельных правок к исходному значению;

6. подмешивание дополнительных фактов из самостоятельного TTL-файла к итоговому графу документа;

7. управление шаблонами извлечения знаний: просмотр кода с подсветкой синтаксиса, запуск структурной валидации с отображением отчёта о замечаниях, открытие кода во внешнем редакторе;

8. автоматизированная генерация заготовки шаблона на основе образца документа и схемы онтологии в многошаговом сценарии;

9. просмотр текущего состояния общей онтологической модели: иерархия классов, перечень индивидов, карточки индивидов и классов с переходами между связанными сущностями;

10. отслеживание происхождения каждого факта в модели: документ, шаблон, эффективная дата, применённая политика слияния, сведения о вытесненных конкурирующих фактах;

11. откат документа — удаление всех ассоциированных с ним фактов из общей модели с автоматической пересборкой состояния.

2 Условия выполнения программы

2.1 Минимальный состав аппаратных и программных средств

- персональный компьютер с процессором уровня не ниже Intel Core i5 / AMD Ryzen 5;
- оперативная память — от 4 ГБ (рекомендуется 8 ГБ);
- дисковое пространство — не менее 1 ГБ свободного объёма (включая виртуальное окружение Python и рабочие данные);
- операционная система — Microsoft Windows 10/11 либо современный дистрибутив Linux;
- интерпретатор Python версии 3.11 (более ранние версии не поддерживаются);
- LibreOffice либо установленный Microsoft Word (для преобразования документов формата DOC);
- сетевое подключение к Интернету для обращения к программному интерфейсу OpenAI и веб-сервису ConvertAPI;
- действующие ключи доступа к указанным внешним сервисам.

2.2 Требования к оператору

В зависимости от выполняемых задач под оператором понимается одна из трёх категорий пользователей:

- **технический специалист** — осуществляет первичное развёртывание системы (создание виртуального окружения Python, установка зависимостей, конфигурация ключей доступа к внешним сервисам, установка LibreOffice), а также сопровождение системы при необходимости обновлений; должен владеть базовыми навыками работы с командной строкой, редактированием конфигурационных файлов и установкой пакетов Python;
- **автор шаблонов извлечения знаний** — разрабатывает шаблоны для новых классов документов кафедры; должен владеть навыками программирования на языке Python, иметь представление о декларативном языке извлечения полей разработанной системы (поле, селектор, извлекатель,

нормализатор), концептах онтологии и построителе графа фактов; знание онтологии кафедры обязательно;

- **оператор обработки документов** — выполняет повседневную обработку документов: загружает документы, запускает конвейер, при необходимости корректирует результат через графический интерфейс, отслеживает состояние общей модели; достаточно общих навыков работы с настольным графическим интерфейсом и понимания предметной области кафедры.

В типичном случае все три роли могут совмещаться одним пользователем.

3 Выполнение программы

3.1 Загрузка и запуск программы

Первичное развёртывание системы выполняет технический специалист в следующей последовательности:

1. установить интерпретатор Python версии 3.11 (если не установлен);
2. в каталоге размещения программы создать виртуальное окружение командой `py -3.11 -m venv .venv`;
3. активировать окружение командой `.venv\Scripts\activate` (Windows) либо `source .venv/bin/activate` (Linux);
4. обновить инструменты сборки командой `python -m pip install --upgrade pip setuptools wheel`;
5. установить зависимости командой `pip install -r requirements.txt`;
6. скопировать файл `.env.example` в `.env` и задать значения переменных `OPENAI_API_KEY` (ключ доступа к программному интерфейсу OpenAI) и `CONVERTAPI_SECRET` (ключ доступа к сервису преобразования PDF);
7. установить LibreOffice (либо убедиться в наличии установленного Microsoft Word);
8. при необходимости настроить выбор интерпретатора в IDE (через `Ctrl + Shift + P → Python: Select Interpreter`).

Запуск программы выполняется командой `python main.py`. После запуска открывается главное окно с тремя тематическими вкладками — *Документы*, *Шаблоны*, *Модель*.

3.2 Типовой сценарий обработки документа

Полный цикл обработки одного документа выполняется в следующем порядке.

1. На вкладке *Документы* нажать кнопку добавления нового документа, выбрать файл в формате DOC, DOCX или PDF; при желании задать осмысленное имя документа.

2. Выбрать загруженный документ в списке слева. В правой части откроется карточка документа с индикатором статуса (документ получает начальный статус «загружен»).

3. Нажать кнопку запуска обработки. Конвейер последовательно выполнит пять стадий, обновляя индикатор статуса. В типовых случаях операция завершается без вмешательства пользователя.

4. По завершении просмотреть результат на трёх суб-вкладках:

- *Оригинал* — предпросмотр первой страницы PDF-копии документа с кнопкой открытия в системной программе;
- *UDDM* — структура унифицированного представления документа в режимах HTML-вид и tree-вид;
- *Граф* — двухпанельный редактор чернового графа фактов с подсветкой неполных и переопределённых элементов.

5. При необходимости отредактировать черновой граф (см. пункт 3.4); ручные правки сохраняются автоматически в отдельном файле.

6. При полном графе нажать кнопку «Добавить в модель» (или просто продолжить запуск конвейера) — извлечённые факты будут слиты с общей онтологической моделью с учётом политик слияния, заданных в схеме.

3.3 Ручное указание класса документа

Если автоматическое определение класса не дало результата (статус документа остановился на стадии «определение класса») или результат оказался ошибочным, пользователь имеет возможность указать класс документа явно:

1. в карточке документа открыть выпадающий список доступных классов (формируется по перечню шаблонов системы);
2. выбрать соответствующий класс;
3. продолжить запуск конвейера — последующие стадии будут использовать указанный пользователем шаблон.

3.4 Ручное редактирование чернового графа фактов

Редактор чернового графа доступен на суб-вкладке *Граф* карточки документа. Левая панель отображает узлы и триплеты графа: полные триплеты — обычным оформлением, неполные — выделены цветом для указания, какие факты не удалось построить автоматически; отредактированные пользователем — отдельным оттенком. Правая панель отображает форму редактирования выбранного элемента.

Доступные операции:

- *исключение триплета* — снимает соответствующий факт из итогового графа документа без удаления из чернового;
- *переопределение узла* — заменяет узел триплета указанным пользователем значением; для узлов, представляющих индивид класса или типизированный литерал, доступна *кнопка генерации значения через концепт*: пользователь выбирает соответствующий концепт онтологии и вводит значение в человекочитаемой форме, а система выполняет полный цикл — разбор, нормализацию, вычисление идентификатора индивида и автоматическое формирование идентифицирующих литералов; перед подтверждением замены показывается *предпросмотр итогового IRI или литерала*;
- *сброс правки* — для каждого ранее отредактированного узла доступна кнопка возврата к исходному значению, рассчитанному автоматически.

При нажатии кнопки «Источник факта» для любого узла открывается диалог с указанием поля шаблона, из которого узел получен; это средство диагностики причин неполноты графа.

3.5 Добавление дополнительных фактов вне шаблона

В тех случаях, когда некоторое знание из документа в принципе не покрывается шаблоном, пользователь может разместить рядом с документом самостоятельный файл `supplementary_facts.ttl` с произвольными RDF-фактами в формате Turtle. При следующем запуске стадии интеграции эти факты будут подмешаны к итоговому графу документа.

3.6 Создание и редактирование шаблонов извлечения

На вкладке *Шаблоны* отображается список шаблонов системы и карточка выбранного шаблона с описанием, исходным кодом (с подсветкой синтаксиса) и сводным отчётом структурной валидации.

Создание нового шаблона возможно в двух режимах:

- **ручное создание** — пользователь нажимает кнопку добавления нового шаблона, задаёт имя и пишет код шаблона во внешнем редакторе по ссылке «Открыть в редакторе»; разработка кода опирается на программный интерфейс ядра системы (концепты, селекторы, извлекатели, нормализаторы, построитель графа);
- **автоматизированная генерация заготовки** — пользователь нажимает кнопку *генерации шаблона*, указывает образец документа из числа уже загруженных; система выполняет два последовательных обращения к большой языковой модели — для формирования таблицы извлекаемых полей и для генерации исполняемого кода по этой таблице. Полученная заготовка открывается в карточке шаблона; ручная доводка выполняется через внешний редактор.

После любого изменения кода шаблона необходимо запустить структурную валидацию (кнопка «Проверить»). Отчёт содержит замечания шести категорий, разделённые по уровням важности «ошибка» и «предупреждение»; ошибки блокируют использование шаблона до их устранения.

3.7 Просмотр онтологической модели

На вкладке *Модель* отображается текущее состояние общей онтологической модели кафедры. Левая панель содержит иерархию классов и перечень индивидов; правая — карточку выбранного объекта.

Особенностью интерфейса является *внутренняя навигация между связанными сущностями*: каждое имя класса или индивида в карточке отображается как гиперссылка; по щелчку выбранный объект становится

текущим, а правая панель открывает его карточку. Это позволяет переходить по связям модели без обращения к глобальному поиску.

Для каждого факта в карточке доступен диалог «Источник факта» с указанием документа-источника, шаблона, эффективной даты документа, применённой политики слияния и сведений о вытесненных конкурирующих фактах. Из диалога возможен прямой переход к карточке документа-источника на вкладке *Документы*.

3.8 Откат и переобработка документа

Откат документа из общей модели выполняется на карточке документа кнопкой «Откатить» — все ассоциированные с документом факты удаляются из общей модели с автоматической пересборкой её состояния из истории документов. Документ остаётся в системе, но переводится в статус «откатан»; для его повторного включения в модель достаточно запустить конвейер с любой стадии.

Переобработка документа (например, после исправления шаблона или загрузки обновлённой версии исходного файла) реализована той же последовательностью «откатить — обработать заново» и поддерживается кнопкой «Сбросить и переобработать»; политики слияния с другими документами применяются заново с возможностью восстановления ранее вытесненных по дате фактов.

3.9 Завершение программы

Завершение работы программы выполняется штатным закрытием главного окна приложения. Все промежуточные результаты и пользовательские правки автоматически сохранены в каталоге *.data/* в персистентном виде и будут доступны при следующем запуске.

4 Сообщения оператору

Программа взаимодействует с оператором через графический интерфейс и журналы. Виды диагностических сообщений сгруппированы следующим образом.

Индикатор статуса документа в верхней части карточки документа показывает достигнутую стадию конвейера и при наличии ошибки — краткое поясняющее сообщение (например, «формат документа не поддерживается», «не удалось определить класс документа», «извлечённый граф неполный»). Кнопки запуска и продолжения обработки становятся доступны или недоступны в зависимости от текущего статуса.

Отчёт структурной валидации шаблона отображается в карточке шаблона; замечания разделены по уровням важности с указанием позиции в исходном коде (номер строки и колонки). Замечания уровня «ошибка» блокируют использование шаблона; замечания уровня «предупреждение» допускают использование, но требуют внимания автора.

Подсказка по опечаткам в обращениях к терминам онтологии — особый случай предупреждения структурной валидации: при обращении к несуществующему термину онтологии система предлагает ближайший по написанию валидный термин.

Подсветка в редакторе графа фактов — визуальное обозначение неполных и переопределённых узлов и триплетов разными оттенками. Для каждого неполного узла доступно пояснение причины и ссылка на поле-источник; для каждого переопределённого — кнопка возврата к исходному значению.

Журнал работы приложения (.data/logs/app.log) — текстовый файл, содержащий записи о каждом запуске стадии конвейера, обо всех событиях слияния с общей моделью, об ошибках и предупреждениях. Предназначен для технического специалиста при расследовании проблем.

Журнал обращений к большой языковой модели (.data/logs/agents.log) — отдельный текстовый файл, содержащий полные тексты системного и

пользовательского промптов, ответ модели и длительность обращения. Позволяет в любой момент восстановить контекст принятого моделью решения.

Отчёт о слиянии документа (`ontology_merge_report.json` в каталоге документа) — структурированный отчёт о фактах, добавленных, заменённых и отклонённых при последнем слиянии документа с общей моделью. Используется при разборе несоответствий ожиданий пользователя и фактического содержания модели.

5 Лист регистрации изменений

Лист регистрации изменений приведён в таблице Д.1.

Таблица Д.1 — Лист регистрации изменений в программном документе «Руководство оператора».

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц) в документе	№ докуме нта	Входящий № сопроводител ьного документа	Подпись	Дата
Номер Изм.	измененных	замененных	новых	аннулированных					